

# TP2 – Algoritmos y estructuras de datos

## Datos personales:

Nombre: Agustin Gomez  
Padrón: 113941  
Email: [agugomez@fi.uba.ar](mailto:agugomez@fi.uba.ar)

Nombre: Agustin Sosa  
Padrón: 114009  
Email: [agsosa@fi.uba.ar](mailto:agsosa@fi.uba.ar)

Nombre: Braian Ramos  
Padrón: 114411  
Email: [bramos@fi.uba.ar](mailto:bramos@fi.uba.ar)

Nombre: Diego Enrique Andrade  
Padrón: 114418  
Email: [deandrade@fi.uba.ar](mailto:deandrade@fi.uba.ar)

Nombre: Joaquin Leonel Orlando  
Padrón: 114650  
Email: [jlorlando@fi.uba.ar](mailto:jlorlando@fi.uba.ar)

## Informe del trabajo realizado:

El trabajo práctico consistió en el desarrollo de AI-Quest, un juego educativo programado en Java cuyo objetivo es recorrer un mapa compuesto por diez ciudades, donde cada ciudad plantea un desafío basado en un algoritmo o una estructura de datos distintos vistos a lo largo de la materia. La idea central fue que el jugador avance de manera progresiva: al completar una ciudad se desbloquea la siguiente, de modo que el recorrido por el mapa funciona también como un recorrido por los distintos temas de la cátedra.

El proyecto se organizó como un trabajo en equipo. Cada integrante se hizo cargo del desarrollo de una o más ciudades, trabajando sobre su propia rama de Git para no interferir con el código de los demás. Una vez que cada ciudad quedaba funcionando de forma independiente, se integraba al proyecto final mediante la fusión de ramas. Este esquema permitió que varias personas trabajaran en paralelo y que la integración se hiciera de forma ordenada, resolviendo los conflictos a medida que aparecían.

La estructura general del proyecto se apoya en tres elementos comunes a todas las ciudades. El primero es el mapa principal, que muestra las diez ciudades y detecta sobre cuál hizo clic el jugador para abrir la ciudad correspondiente. El segundo es el sistema de progreso, encargado de registrar qué ciudades están desbloqueadas, de guardar el avance en disco y de cargarlo nuevamente al retomar una partida. Cada ciudad, al ser superada, marca como desbloqueada a la siguiente y guarda ese progreso, de manera que el estado del juego persiste entre sesiones. El tercer elemento es un conjunto de utilidades compartidas, que reúne validaciones y funciones auxiliares reutilizadas por todas las ciudades.

En cuanto a la implementación, todo el juego está desarrollado en Java, utilizando la biblioteca Swing para las interfaces gráficas. La representación visual de los algoritmos se resolvió generando imágenes a partir del estado de cada estructura, lo que permite mostrar el funcionamiento paso a paso en lugar de limitarse a un resultado final. El progreso del jugador se conserva mediante la serialización del estado del juego a un archivo, que luego puede volver a cargarse. Se trabajó siempre con rutas relativas, de modo que el proyecto pueda ejecutarse en cualquier equipo sin necesidad de modificar el código.

Cada ciudad aborda un tema diferente. La Ciudad 1 plantea la exploración de una matriz tridimensional, donde el jugador recorre una cueva de varios pisos recolectando objetos. La Ciudad 2 resuelve el problema de las N reinas mediante backtracking, mostrando cómo el algoritmo prueba posiciones y retrocede cuando un camino no lleva a una solución. La Ciudad 3 utiliza también backtracking, esta vez para resolver un laberinto mediante búsqueda en profundidad. La Ciudad 4 visualiza dos algoritmos de ordenamiento, Bubble Sort y Quick Sort, representando el arreglo con barras. La Ciudad 5 trabaja con técnicas de búsqueda sobre estructuras de datos. La Ciudad 6 implementa una tabla hash con encadenamiento separado, mostrando el cálculo de la función hash, las colisiones y el recorrido de las cadenas. La Ciudad 7 es un gestor de grafos que permite crear nodos y aristas y ejecutar algoritmos clásicos como flujo máximo y camino mínimo. La Ciudad 8 resuelve las Torres de Hanoi mediante recursión, animando el movimiento de los discos. La Ciudad 9 implementa un sistema de combate por turnos que pone en práctica el uso de listas, colas y pilas. Finalmente, la Ciudad 10 aborda el análisis de recurrencias.

A lo largo del desarrollo, una de las principales dificultades fue la integración de las distintas ciudades en un único proyecto, ya que cada una había sido pensada y programada de manera independiente. Unificar el manejo del progreso, la navegación entre el mapa y cada ciudad, y el retorno al mapa una vez completado cada desafío, requirió coordinar las decisiones tomadas por cada integrante. El uso de ramas de Git fue clave para sostener este proceso, permitiendo desarrollar en paralelo y luego integrar de forma controlada.

Como resultado, se obtuvo un juego funcional que recorre los principales temas de la materia de una manera interactiva, donde cada ciudad no solo presenta un desafío jugable sino que además muestra visualmente el funcionamiento interno del algoritmo o la estructura de datos que la sustenta.

## Cuestionario

### 1) ¿Qué es un svn?

SVN (Subversion) es un sistema de control de versiones centralizado. Su función es llevar un registro de todos los cambios que se realizan sobre los archivos de un proyecto a lo largo del tiempo, permitiendo que varias personas trabajen sobre el mismo código sin pisar el trabajo de los demás.

Al ser centralizado, existe un único repositorio principal que se encuentra en un servidor. Cada integrante descarga una copia de los archivos desde ese servidor, realiza sus modificaciones y luego las envía nuevamente al repositorio central.

De esta manera, SVN permite mantener un historial completo del proyecto, volver a versiones anteriores, saber quién realizó cada cambio y resolver los conflictos que surgen cuando dos personas modifican el mismo archivo.

### 2) ¿Que es una “Ruta absoluta” o una “Ruta relativa”?

Una ruta absoluta es la dirección completa de un archivo o carpeta partiendo desde la raíz del sistema de archivos. Indica la ubicación exacta del archivo sin depender de la posición desde la que se ejecute el programa. Por ejemplo: C:\Users\usuario\Tp2\src\main.java.

Una ruta relativa, en cambio, indica la ubicación de un archivo respecto a la carpeta desde la cual se está ejecutando el programa. No parte desde la raíz, sino desde el directorio actual. Por ejemplo: \src\main.java.

La diferencia principal es que una ruta absoluta solo funciona en la computadora en la que fue escrita, mientras que una ruta relativa funciona en cualquier equipo, siempre que se respete la estructura interna de carpetas del proyecto. Por este motivo, en el trabajo se utilizan rutas relativas, de modo que el programa pueda ejecutarse correctamente en cualquier máquina sin necesidad de modificar el código.

### 3) ¿Qué es git?

Git es un sistema de control de versiones distribuido. Al igual que SVN, permite registrar el historial de cambios de un proyecto y facilitar el trabajo en equipo, pero se diferencia en que no depende de un único servidor central.

En Git, cada integrante posee una copia completa del repositorio en su propia computadora, incluyendo todo el historial de cambios. Esto permite trabajar de forma local, realizar cambios y luego sincronizarlos con un repositorio remoto (por ejemplo, en GitHub) mediante operaciones como push y pull.

Otra característica importante de Git es el manejo de ramas, que permiten que cada integrante desarrolle una funcionalidad por separado y luego una su trabajo con el del resto mediante una fusión. En este trabajo se utilizó Git para que cada integrante desarrollara su ciudad en una rama propia y posteriormente integrar todas las ciudades en el proyecto final.

## Manual de usuario

# Manual del Usuario – Menú Principal y Mapa General (AI-Quest)

## Objetivo

El paquete principal de AI-Quest permite administrar las partidas del jugador y acceder al mapa general del juego. Desde aquí es posible crear una nueva partida, cargar una partida existente, eliminar partidas guardadas y navegar entre las distintas ciudades disponibles.

---

## Inicio de la aplicación

Al ejecutar el programa se abre automáticamente el menú principal.

La ventana presenta cuatro opciones:

- **Iniciar Partida**
  - **Cargar Partida**
  - **Borrar Partida**
  - **Salir**
- 

## Iniciar Partida

Al seleccionar **Iniciar Partida**:

1. Se solicita el nombre del jugador.
2. Se crea una nueva partida.
3. El progreso inicial se guarda automáticamente.
4. Se abre el mapa principal del juego.

Al comenzar una nueva partida, únicamente estarán disponibles las ciudades desbloqueadas por defecto.

---

## Cargar Partida

La opción **Cargar Partida** permite continuar una partida guardada anteriormente.

Funcionamiento:

1. El sistema muestra una lista con todas las partidas disponibles.
2. El jugador selecciona una partida.

3. Se carga el progreso almacenado.
4. Se abre el mapa con las ciudades desbloqueadas previamente.

Si no existen partidas guardadas, el sistema mostrará un mensaje informativo.

---

## Borrar Partida

La opción **Borrar Partida** permite eliminar una partida almacenada.

Pasos:

1. Se muestra la lista de partidas guardadas.
2. El jugador selecciona la partida a eliminar.
3. El sistema borra el archivo correspondiente.
4. Se muestra un mensaje confirmando la eliminación.

Si no existen partidas guardadas, se informará al usuario.

---

## Salir

El botón **Salir** cierra completamente la aplicación.

---

## Mapa General

Una vez iniciada o cargada una partida se accede al mapa principal del juego.

El mapa contiene las diez ciudades de Al-Quest distribuidas visualmente sobre una imagen interactiva.

Para ingresar a una ciudad:

1. Hacer clic sobre su ubicación en el mapa.
  2. Si la ciudad está desbloqueada, se abrirá automáticamente.
  3. Si la ciudad aún no está disponible, no se realizará ninguna acción.
- 

## Sistema de Progresión

Cada ciudad completada desbloquea nuevas ciudades.

El progreso se almacena automáticamente en la partida del jugador.

Al volver a cargar una partida:

- Se conservan las ciudades desbloqueadas.
  - Se mantiene el avance realizado.
  - No es necesario repetir ciudades ya completadas.
- 

## Ciudades Disponibles

El mapa permite acceder a las siguientes ciudades:

| Ciudad | Temática |
|--------|----------|
|--------|----------|

|          |              |
|----------|--------------|
| Ciudad 1 | Introducción |
|----------|--------------|

|          |              |
|----------|--------------|
| Ciudad 2 | Recursividad |
|----------|--------------|

|          |       |
|----------|-------|
| Ciudad 3 | Pilas |
|----------|-------|

|          |               |
|----------|---------------|
| Ciudad 4 | Ordenamientos |
|----------|---------------|

|          |           |
|----------|-----------|
| Ciudad 5 | Búsquedas |
|----------|-----------|

|          |         |
|----------|---------|
| Ciudad 6 | Árboles |
|----------|---------|

|          |        |
|----------|--------|
| Ciudad 7 | Grafos |
|----------|--------|

|          |                 |
|----------|-----------------|
| Ciudad 8 | Torres de Hanoi |
|----------|-----------------|

|          |                       |
|----------|-----------------------|
| Ciudad 9 | Combate por<br>turnos |
|----------|-----------------------|

|              |               |
|--------------|---------------|
| Ciudad<br>10 | Desafío Final |
|--------------|---------------|

Cada ciudad presenta una actividad relacionada con estructuras de datos o algoritmos vistos durante el recorrido del juego.

---

## Guardado del Progreso

El progreso del jugador se almacena mediante archivos de partida.

Se guardan:

- Nombre del jugador.
- Ciudades desbloqueadas.
- Estado general del avance.

Los datos permanecen disponibles incluso después de cerrar la aplicación.

---

## Recomendaciones de uso

- Utilizar nombres de jugador válidos al crear una partida.
  - Completar las ciudades en orden para avanzar naturalmente en la progresión.
  - No eliminar archivos de la carpeta de partidas manualmente.
  - Cargar siempre la partida correspondiente para conservar el avance obtenido.
- 

## Finalización del juego

El objetivo principal consiste en completar todas las ciudades del mapa hasta alcanzar la **Ciudad 10**, donde se encuentra el desafío final de Al-Quest.

Al completar cada ciudad se desbloquearán nuevas zonas hasta recorrer la totalidad del juego.

## Ciudad 1: Recolección en Matriz

### Objetivo

La ciudad 1 consiste en explorar una cueva de tres pisos en busca de tres elementos importantes:

- Antorcha Perdida
- Mapa Rasgado
- Amuleto Místico



El jugador deberá recorrer los distintos pisos, abrir cofres y usar los elementos que pueden brindarle alguna ayuda para localizar el resto de estos mismos. Una vez reunidos los tres elementos, la ciudad se considera completada.

## **Inicio de la aplicación**

Al ingresar a la Ciudad 1 se muestra una pantalla con la cueva vista desde arriba.

La interfaz contiene:

- El mapa del piso actual.
- El personaje controlado por el jugador.
- Cofres distribuidos por la mazmorra.
- Escaleras para subir/bajar de piso.
- Una mochila con tres espacio para objetos.
- Un área de mensajes donde se muestran eventos y pistas.

## **Controles**

Movimiento e Interacción:

- **W**: mover hacia arriba.
- **A**: mover hacia la izquierda.
- **S**: mover hacia abajo.
- **D**: mover hacia la derecha.
- **E**: Interacción con cofres/escaleras.

Uso de objetos:

- **1**: usar el objeto del primer espacio de la mochila.
- **2**: usar el objeto del segundo espacio de la mochila.
- **3**: usar el objeto del tercer espacio de la mochila.

## **Exploración**

La cueva posee tres pisos

Cada vez que se inicia una partida se generan variantes diferentes de cada piso modificando la ubicación de:

- Muros
- Escaleras
- Cofres

Esto provoca que cada partida tenga una distribución diferente.

## **Cofres**

Al caminar sobre un cofre el jugador puede abrirlo.

Los cofres puede contener:

- Elemento.
- Trampa.
- Ningún Objeto.

## **Elementos**

Antorcha Perdida

Aumenta la visibilidad del jugador de forma pasiva en un casillero.

Mapa Rasgado

Revela una pista sobre la ubicación aproximada del Amuleto Místico dentro del tercer piso.

Amuleto Místico

Es el objeto más útil del juego, te permite hacer cosas maravillosas, como saber tu posición en cada momento.

## **Trampas**

Portal Inestable

Teletransporta al jugador al primer piso.

Interferencia

Reduce la visibilidad del jugador a una casilla

## **Victoria**

La ciudad se completa cuando el jugador obtiene los tres elementos.  
Al conseguirlos aparece un mensaje de felicitación.

## **Mensajes**

Durante la exploración se muestran mensaje informativos sobre:

- Apertura de cofres
- Obtención de objetos
- Uso de objetos
- Pistas del mapa rasgado.
- Victoria de la partida.

## Ciudad 2: Problema de las N Reinas

### Objetivo

La Ciudad 2 permite visualizar la resolución del clásico problema de las N Reinas: ubicar N reinas en un tablero de  $N \times N$  sin que ninguna ataque a otra, es decir, sin que compartan fila, columna ni diagonal. El usuario elige la cantidad de reinas y la posición de la primera, y observa cómo el algoritmo de backtracking prueba posiciones y retrocede cuando un camino no lleva a una solución.

### Inicio de la aplicación

La ciudad comienza con una pantalla de presentación que explica de qué se trata y cómo ganarla. Al presionar el botón Comenzar se pasa a la pantalla de juego, donde se muestran los siguientes componentes:

- Un selector (Cantidad de reinas) para elegir cuántas reinas usar, entre 1 y 8.
- Dos selectores (Fila inicial y Columna inicial) para indicar dónde se coloca la primera reina.
- Un botón Resolver que ejecuta el algoritmo.
- Un área gráfica donde se dibuja el tablero con las reinas.
- Un slider y los botones Anterior y Siguiente para recorrer el paso a paso.
- Un botón Salir a las ciudades, que aparece recién al ganar la ciudad.

### Ingreso de datos

El usuario primero elige la cantidad de reinas. Al cambiar ese valor, el rango de la fila y la columna iniciales se ajusta automáticamente para que siempre queden dentro del tablero. Luego se elige la fila y la columna donde irá la primera reina.

### Ejecución

Al presionar Resolver:

Se crea un tablero del tamaño elegido y se coloca la reina inicial en la posición indicada.

El algoritmo de backtracking ubica las reinas restantes, guardando un paso por cada colocación y cada retroceso.

Se generan las imágenes del tablero para cada paso y se habilita la navegación.

Las reinas colocadas se ven en dorado; cuando un paso corresponde a un retroceso, la reina que se quita se marca en rojo.

Debajo del tablero, un cartel informa si se encontró solución y en cuántos pasos, o si no existe solución para esa configuración inicial.

### Desbloqueo de la siguiente ciudad

Para ganar la Ciudad 2 hay que encontrar una posición inicial con la que el backtracking llegue a la solución en menos de 10 pasos. Cuanto mejor se elija el arranque, menos retrocesos necesita el algoritmo. Al lograrlo, el sistema muestra un mensaje de felicitación, marca la ciudad como ganada y habilita el botón para salir al mapa de ciudades.

## Botón Salir

El botón Salir a las ciudades cierra la Ciudad 2 y regresa al mapa del juego según la integración general.

## Manejo de errores

Los datos se ingresan mediante selectores numéricos con rangos controlados, por lo que el usuario no puede escribir valores inválidos. Si para una configuración no existe solución, el programa lo informa con un cartel en rojo y no marca la ciudad como ganada.

## Ciudad 3: Laberinto Hacker (Backtracking)

### Objetivo

La Ciudad 3 está pensada para mostrar cómo funciona el algoritmo de **Backtracking (DFS o búsqueda en profundidad)** resolviendo un laberinto con una ambientación tipo *hackeo*. La idea es que el usuario pueda ver el recorrido del algoritmo mientras prueba distintos caminos, vuelve atrás cuando queda bloqueado y sigue buscando hasta encontrar la salida o concluir que el laberinto no tiene solución.

### Inicio de la aplicación

Cuando el jugador entra a esta ciudad desde el mapa principal, primero aparece una pantalla intermedia con dos opciones: **Iniciar el Protocolo** o **Volver al Mapa**.

Si se inicia el protocolo, se abre una ventana aparte donde aparecen:

- Un panel principal que muestra el laberinto, diseñado con una estética inspirada en circuitos y memorias RAM.
- La casilla inicial, marcada visualmente en verde, y el punto de llegada.
- Un panel inferior con tres botones: **Empezar Hackeo**, **Abortar** y **Salir**.

### Funcionamiento

Al presionar **Empezar Hackeo**, ocurre lo siguiente:

1. Se carga un laberinto aleatorio.
2. El algoritmo comienza desde la posición inicial **(0,0)** y va marcando el camino recorrido con una estela amarilla.
3. Si llega a un camino sin salida, retrocede sobre sus pasos y prueba otra alternativa.

4. Si encuentra la salida, aparece un mensaje indicando “**Sistemas Hackeados con Éxito**”, acompañado por una imagen temática. En caso de no existir una solución, se muestra una alerta de “**Antivirus Activado**”.

### Botón Abortar

Si el usuario quiere detener la ejecución, puede usar el botón **Abortar**. Esto interrumpe el algoritmo en ese momento y deja el recorrido marcado hasta donde llegó. Además, aparece un mensaje de advertencia indicando que el proceso fue cancelado, permitiendo volver a intentarlo sin reiniciar la aplicación.

### Desbloqueo de la siguiente ciudad

Para completar la Ciudad 3, es necesario ejecutar el hackeo y lograr que el algoritmo encuentre una salida al menos una vez. Cuando esto sucede, el sistema guarda el progreso, muestra un mensaje de felicitación y habilita la **Ciudad 4**.

### Botón Salir

El botón **Salir** cierra la ventana del laberinto y vuelve a mostrar el mapa principal del juego.

### Manejo de errores

Se agregó un control para evitar que se ejecuten varios procesos al mismo tiempo. Si el usuario presiona **Empezar Hackeo** mientras el algoritmo ya está corriendo, la acción se ignora mediante una bandera de control **hackeoEnProgreso**, evitando errores visuales o comportamientos inesperados. Para empezar de nuevo, debe terminar el laberinto con o sin éxito, o cancelar la ejecución con el botón **Abortar**.

## Ciudad 4: Ordenamientos

### Objetivo

La Ciudad 4 permite visualizar el funcionamiento de dos algoritmos clásicos de ordenamiento: **Bubble Sort** y **Quick Sort**. El usuario puede ingresar una secuencia de números y observar gráficamente cómo el algoritmo los ordena paso a paso mediante una animación.

### Inicio de la aplicación

Al abrir la ventana se muestran los siguientes componentes:

- Un campo de texto para ingresar los números.
- Un selector para elegir el algoritmo de ordenamiento.
- Un botón Ordenar para iniciar la ejecución.

- Un botón Salir para cerrar la ventana.
- Un área gráfica donde se representan los valores mediante barras verticales.

## Ingreso de datos

Los números deben escribirse separados por comas.

Ejemplos válidos:

- 5, 3
- 10, 7, 4, 9, 2

No deben ingresarse caracteres que no sean números o comas.

## Selección del algoritmo

El usuario puede elegir entre:

- **BubbleSort:** compara elementos adyacentes e intercambia sus posiciones cuando están desordenados hasta obtener el arreglo ordenado.
- **QuickSort:** selecciona un pivote y reorganiza los elementos en torno a él mediante particiones recursivas.

## Ejecución

Al presionar **Ordenar**:

1. Se leen los números ingresados.
2. Se ejecuta el algoritmo seleccionado.
3. Se almacenan los estados intermedios del vector.
4. Se reproduce una animación mostrando cada paso del proceso mediante barras de diferentes alturas.

Cada barra representa un número y su altura es proporcional a su valor.

## Desbloqueo de la siguiente ciudad

Para completar la Ciudad 4 es necesario ejecutar ambos algoritmos al menos una vez:

- Bubble Sort.
- Quick Sort.

Cuando se hayan utilizado los dos, el sistema mostrará un mensaje de felicitación y desbloqueará automáticamente la Ciudad 5, guardando dicho progreso.

## Botón Salir

El botón Salir cierra la ventana de la Ciudad 4 y finaliza la interacción con este módulo, regresando al menú correspondiente según la integración del juego.

## Manejo de errores

Si los datos ingresados no tienen un formato válido (por ejemplo, contienen letras o símbolos no permitidos), el programa no podrá convertirlos a números y la operación de ordenamiento fallará. Se recomienda ingresar únicamente números enteros separados por comas.

## Ciudad 5: Búsquedas y Comparación de Algoritmos

### Objetivo

La Ciudad 5 permite comparar dos métodos de búsqueda de palabras dentro de un archivo de texto:

- Búsqueda Lineal.
- Árbol Binario de Búsqueda (ABB).

El jugador podrá cargar un archivo, buscar palabras y observar las diferencias de rendimiento entre ambos algoritmos.

### Inicio de la aplicación

Al abrir la ventana se muestran los siguientes componentes:

- Un botón **Cargar TXT** para seleccionar un archivo de texto.
- Un campo de texto para ingresar la palabra a buscar.
- Un botón **Buscar** para ejecutar la búsqueda.
- Un botón **Salir** para cerrar la ventana.
- Un área de resultados donde se muestran los datos obtenidos por cada algoritmo.

### Carga de archivo

Antes de realizar búsquedas es necesario cargar un archivo de texto.

Al presionar **Cargar TXT**:

- Se abre un explorador de archivos.
- El usuario selecciona un archivo **.txt**.
- El sistema procesa todas las palabras del archivo.
- Las palabras se almacenan simultáneamente en:
  - una estructura de búsqueda lineal;
  - un árbol binario de búsqueda.

Una vez cargado correctamente, el sistema mostrará un mensaje de confirmación.

## Búsqueda de palabras

El usuario debe escribir una palabra en el campo correspondiente y presionar **Buscar**.

Ejemplos:

java

algoritmo

estructura

La búsqueda se realiza utilizando ambas estructuras de datos.

## Resultados mostrados

Para cada algoritmo se informa:

- Línea donde se encontró la palabra.
- Posición de la palabra dentro de la línea.
- Tiempo de ejecución medido en nanosegundos.
- Cantidad de operaciones realizadas.

Además, el sistema muestra una comparación indicando:

- Qué algoritmo fue más rápido.
- Qué algoritmo realizó menos operaciones.

## Búsqueda Lineal

La búsqueda lineal recorre secuencialmente todas las palabras almacenadas hasta encontrar la palabra solicitada.

Características:

- Implementación simple.
- Puede requerir muchas comparaciones.
- Su rendimiento disminuye a medida que aumenta el tamaño del archivo.

## Árbol Binario de Búsqueda (ABB)

El ABB organiza las palabras siguiendo un criterio de orden alfabético.

Características:

- Permite descartar grandes porciones de datos durante la búsqueda.
- Generalmente realiza menos operaciones.



- Suele obtener mejores tiempos de ejecución que la búsqueda lineal.

## Desbloqueo de la siguiente ciudad

Para completar la Ciudad 5 es necesario:

1. Cargar correctamente un archivo de texto.
2. Realizar al menos **3 búsquedas**.

Cuando ambas condiciones se cumplen:

- El sistema mostrará un mensaje de felicitación.
- Se desbloqueará automáticamente la **Ciudad 6**.
- El progreso quedará guardado en la partida.

## Botón Salir

El botón **Salir** cierra la ventana de la Ciudad 5 y finaliza la interacción con este módulo, regresando al menú correspondiente según la integración del juego.

## Manejo de errores

- Debe cargarse un archivo válido antes de realizar búsquedas.
- La palabra buscada no puede estar vacía.
- Si la palabra no existe en el archivo, el sistema indicará que no fue encontrada.
- Si ocurre un problema durante la carga del archivo, se mostrará un mensaje de error correspondiente.

## Ciudad 6: Hashing

### Objetivo

La Ciudad 6 permite visualizar el funcionamiento de una tabla hash con direccionamiento por encadenamiento separado. El usuario inserta y busca palabras y observa paso a paso cómo la función hash decide en qué bucket se guarda cada una, qué pasa cuando dos palabras caen en el mismo bucket (una colisión) y cómo se recorre la cadena al buscar.

### Inicio de la aplicación

- Al abrir la ventana se muestran los siguientes componentes:
- Un campo de texto (Palabra) para escribir la palabra a insertar o buscar.
- Un botón Insertar y un botón Buscar.

- Una etiqueta de progreso que indica cuántos buckets cumplen el objetivo.
- Un área gráfica donde se dibuja la tabla con sus buckets y cadenas.
- Un slider y los botones Anterior y Siguiente para recorrer los pasos de cada operación.
- Un área de texto inferior con la explicación de cada paso.
- Un botón Pasar a la siguiente ciudad, que aparece recién al ganar.

## Ingreso de datos

Se escribe una palabra en el campo de texto y se presiona Insertar o Buscar. La palabra no puede estar vacía. La función hash usa la suma de los valores ASCII de los caracteres de la palabra, módulo la cantidad de buckets de la tabla (7), para decidir el bucket.

## Ejecución

Al insertar una palabra:

- Se calcula la suma ASCII y el índice del bucket.
- Se revisa la cadena de ese bucket para evitar duplicados (la clave es única).
- Si el bucket estaba vacío, la palabra se inserta directo; si ya tenía elementos, hay una colisión y se encadena al final.

Al buscar una palabra se recorre la cadena del bucket correspondiente comparando una a una hasta encontrarla o agotar la cadena. Además, al buscar se muestra el tiempo real de la búsqueda y la cantidad de comparaciones realizadas. Cada paso se ve resaltado con un color según su tipo (cálculo, comparación, colisión, encontrado, no encontrado).

## Desbloqueo de la siguiente ciudad

Para ganar la Ciudad 6 hay que llenar al menos 5 de los 7 buckets con 2 palabras o más cada uno. La etiqueta de progreso se actualiza con cada inserción. Al alcanzar el objetivo, el sistema muestra un mensaje de felicitación, desbloquea y guarda el avance de la Ciudad 7 y habilita el botón para continuar.

## Botón Salir

El botón Pasar a la siguiente ciudad ejecuta la acción definida por el juego (por ejemplo, volver al mapa). En la prueba independiente de la ciudad, cierra el programa.

## Manejo de errores

Si el campo de texto está vacío al insertar o buscar, el programa no realiza la operación y muestra el error en el área de explicación, sin cortar la ejecución.

# Ciudad 7: Gestor de Grafos

## Objetivo

La Ciudad 7 tiene como objetivo permitir la creación y manipulación de **grafos**. El usuario puede agregar nodos, conectarlos mediante aristas y luego probar distintos algoritmos clásicos de grafos, como **Caminos Mínimos** para encontrar la ruta más corta o **Flujo Máximo** para calcular la capacidad de una red.

## Inicio de la aplicación

Al entrar a la Ciudad 7 desde el mapa principal, aparece un menú intermedio llamado **“Menú - Ciudad 7 (Grafos)”**, donde el jugador puede elegir entre iniciar el programa o volver al mapa.

Al seleccionar **“Iniciar Programa”**, se abre una ventana en pantalla completa con fondo negro, simulando una pizarra. En la parte inferior se encuentra un menú con las herramientas necesarias para trabajar sobre el grafo.

## Ingreso de datos

### Nodos

Los nodos se agregan mediante una ventana emergente donde únicamente se solicita el nombre.

### Aristas

Las conexiones entre nodos se crean desde un panel donde el usuario selecciona:

- nodo de origen;
- nodo de destino;
- tipo de conexión (dirigida o no dirigida);
- dirección del flujo;
- valor de **masa**, utilizado como peso o capacidad de la conexión.

La masa debe ingresarse como un número entero positivo.

## Funcionamiento principal:

### Agregar Nodo

Crea un nuevo nodo representado por un círculo azul con el nombre ingresado.

### Conectar Nodos

Permite unir nodos mediante líneas o flechas, dependiendo del tipo de conexión elegido. En el caso de conectar un nodo consigo mismo, se dibuja un bucle alrededor de él.

### Eliminar Arista

Permite quitar una conexión existente entre dos nodos seleccionados.

### Eliminar Nodo

Elimina un nodo del grafo y también todas las aristas asociadas a ese nodo.

### Calcular Flujo Máximo

Solicita un nodo **Fuente** y un **Sumidero**. Luego de realizar el cálculo, el programa muestra la capacidad máxima alcanzable y marca en rojo las aristas utilizadas durante el recorrido del flujo.

### Calcular Caminos Mínimos

Pide un nodo de origen y uno de destino. Si existe una ruta válida, el sistema muestra el recorrido más conveniente y resalta en verde tanto los nodos como las aristas involucradas.

### Desbloqueo de la siguiente ciudad

Para completar esta ciudad, el jugador debe usar al menos una vez todas las herramientas disponibles del menú inferior. Cuando se utiliza la última opción pendiente, el sistema registra el progreso, muestra un mensaje de felicitación y desbloquea la **Ciudad 8**.

### Botón Salir

El botón **Salir** cierra la ventana de grafos y vuelve a mostrarse el mapa principal del juego.

### Manejo de errores

Se agregaron distintas validaciones para evitar errores durante el uso del programa:

- Para la mayoría de las opciones que puede elegir el usuario, el grafo no debe estar vacío.
- No se permite crear nodos con nombres repetidos.
- Las aristas no pueden tener valores negativos ni texto; solo se aceptan números enteros mayores a cero.
- Si se intenta eliminar una arista inexistente, el sistema muestra un aviso y cancela la operación.
- En el cálculo de **Flujo Máximo**, se controla que la **Fuente** y el **Sumidero** no sean el mismo nodo.
- En **Caminos Mínimos**, se informa al usuario cuando no existe un camino posible entre el origen y el destino.

## Ciudad 8: Torres de Hanoi

### Objetivo

La Ciudad 8 tiene como objetivo demostrar el funcionamiento del algoritmo recursivo de las Torres de Hanoi mediante una animación gráfica.

El jugador puede observar cómo el algoritmo resuelve automáticamente el problema trasladando todos los discos desde la torre de origen hasta la torre destino, respetando las reglas del juego.

---

## Inicio de la aplicación

Al abrir la ventana se muestran los siguientes componentes:

- Una pantalla inicial con el título de la ciudad.
  - Un botón **Iniciar Hanoi** para comenzar la simulación.
  - Un botón **Salir** para cerrar la ventana.
  - Un área gráfica donde se visualizarán las torres y los discos.
- 

## Funcionamiento del problema

La simulación utiliza tres torres:

- Torre de origen.
- Torre auxiliar.
- Torre de destino.

Inicialmente todos los discos se encuentran apilados en la torre de origen, ordenados de mayor a menor tamaño.

El objetivo consiste en mover todos los discos hasta la torre destino utilizando la torre auxiliar como apoyo.

---

## Reglas de las Torres de Hanoi

Durante toda la ejecución se respetan las siguientes reglas:

1. Solo puede moverse un disco por vez.
  2. Solo puede moverse el disco que se encuentra en la parte superior de una torre.
  3. Nunca puede colocarse un disco grande sobre uno más pequeño.
- 

## Ejecución

Al presionar **Iniciar Hanoi**:

1. Se generan tres torres.
2. Se colocan cuatro discos en la torre de origen.
3. Se ejecuta el algoritmo recursivo de Torres de Hanoi.
4. Se calculan todos los movimientos necesarios para resolver el problema.
5. Los movimientos se reproducen mediante una animación gráfica.
6. Cada movimiento actualiza la posición de los discos en pantalla.

La animación continúa hasta completar la resolución del problema.

---

## Representación gráfica

La simulación utiliza una representación visual compuesta por:

- Tres postes verticales que representan las torres.
- Una base horizontal.
- Discos de diferentes tamaños representados mediante rectángulos.

Cada disco posee un ancho proporcional a su tamaño:

- Los discos más grandes tienen mayor ancho.
- Los discos más pequeños tienen menor ancho.

Durante la animación los discos cambian de torre siguiendo exactamente los movimientos calculados por el algoritmo.

---

## Algoritmo utilizado

La resolución se realiza mediante recursividad.

Para mover una torre de  $n$  discos:

1. Se mueven los primeros  $n-1$  discos a la torre auxiliar.
2. Se mueve el disco más grande a la torre destino.
3. Se trasladan los  $n-1$  discos restantes desde la torre auxiliar hasta la torre destino.

Este procedimiento se repite hasta llegar al caso base de un único disco.

---

## Finalización de la ciudad

Cuando el algoritmo completa todos los movimientos:

- Se muestra un mensaje de felicitación.
  - La ciudad se considera completada.
  - Se desbloquea automáticamente la siguiente ciudad.
  - El progreso se guarda en la partida actual.
- 

## Botón Salir

El botón **Salir** cierra la ventana de la Ciudad 8 y finaliza la interacción con este módulo, regresando al menú correspondiente según la integración del juego.

---

## Manejo de errores

El sistema incorpora validaciones para evitar estados inválidos durante la ejecución:

- La cantidad de discos debe ser mayor que cero.
- Las referencias a las torres no pueden ser nulas.
- Los índices de origen, destino y auxiliar deben pertenecer al rango válido de torres.
- Los elementos gráficos utilizados para el dibujo deben existir correctamente antes de ser utilizados.

## Ciudad 9: Sistema de Combate por Turnos

### Descripción General

La Ciudad 9 consiste en un sistema de combate por turnos en el que el jugador debe derrotar a un grupo de enemigos utilizando distintas acciones.

Durante la partida, el usuario puede seleccionar acciones para el jugador y luego procesar los turnos del combate. El estado de la batalla se visualiza gráficamente mediante una interfaz basada en bitmap.

### Acceso a la Ciudad

1. Iniciar la aplicación.
2. Desde el mapa principal seleccionar la Ciudad 9.
3. Se abrirá la ventana correspondiente al combate.

## Interfaz

La ventana de la Ciudad 9 está compuesta por:

### Área de combate

Muestra:

- Nombre del jugador.
- Vida actual del jugador.
- Lista de enemigos.
- Vida actual de cada enemigo.

### Botones de acción

| Botón          | Función                                                               |
|----------------|-----------------------------------------------------------------------|
| Ataque         | Agrega una acción de ataque a la pila de acciones.                    |
| Defensa        | Agrega una acción defensiva que recupera vida del jugador.            |
| Habilidad      | Agrega una acción especial que causa mayor daño que un ataque normal. |
| Procesar Turno | Ejecuta las acciones acumuladas y avanza el combate.                  |
| Salir          | Cierra la ventana de la Ciudad 9.                                     |

## Mecánica del Combate

El combate se desarrolla por turnos.

**Ataque:** Produce daño sobre el primer enemigo vivo encontrado.

**Defensa:** Permite recuperar puntos de vida.

**Habilidad:** Produce un daño superior al ataque normal.

## Ejecución de Acciones

Las acciones seleccionadas por el usuario se almacenan temporalmente antes de ejecutarse.

Ejemplo:



1. Presionar "Ataque".
2. Presionar "Defensa".
3. Presionar "Habilidad".
4. Presionar "Procesar Turno".

Las acciones serán ejecutadas durante el turno del jugador según la lógica implementada por el sistema.

### **Condiciones de Victoria**

El jugador gana cuando:

- Todos los enemigos han sido derrotados.
- El jugador continúa con vida.

Al finalizar la batalla se mostrará un mensaje de victoria.

### **Condiciones de Derrota**

El jugador pierde cuando:

- Su vida llega a cero.

Al finalizar la batalla se mostrará un mensaje de derrota.

### **Ejemplo de Uso**

1. Abrir la Ciudad 9 desde el mapa principal.
2. Observar la lista de enemigos disponibles.
3. Seleccionar una o más acciones.
4. Presionar "Procesar Turno".
5. Repetir el proceso hasta eliminar todos los enemigos o ser derrotado.

### **Consideraciones**

- La ventana puede cerrarse en cualquier momento mediante el botón "Salir".
- El estado del combate se actualiza automáticamente después de cada turno.
- Una vez finalizado el combate, las acciones quedan deshabilitadas y no pueden seguir utilizándose.

## Ciudad 10: Complejidad Algorítmica

### Objetivo

La Ciudad 10 permite analizar recurrencias de la forma:

$T(n)=aT(n/b)+n$  o  $T(n)=aT(n/b)+n^k$

mediante el uso del Teorema Maestro

El usuario debe ingresar distintas recurrencias, observar su expansión y determinar los tres casos del Teorema Maestro. Al descubrir los tres casos, la ciudad queda completada.

### Inicio de la aplicación

La ciudad comienza mostrando una pantalla con:

- Un área de instrucciones.
- Un campo de texto para ingresar la recurrencia.
- Un botón “Analizar”.
- Un panel de resultados.
- Un gráfico que representa el árbol de recurrencia.

### Ingreso de datos

El usuario debe escribir una recurrencia respetando alguno de los siguientes formatos:

$T(n)=aT(n/b)+n$

$T(n)=aT(n/b)+n^k$

Por ejemplo:

$T(n)=2T(n/2)+n$

$T(n)=4T(n/2)+n$

$T(n)=8T(n/4)+n^3$

El valor de “a” debe ser mayor que 0.

El valor de “b” debe ser mayor que 1.

Si la expresión no cumple el formato esperado, el sistema mostrará un mensaje de error.

## Ejecución

Al presionar el botón “Analizar”:

1. Se verifica que la recurrencia tenga un formato válido.
2. Se extraen los valores de: “a”, “b” y “k”
3. Se calcula el valor de: “log<sub>b</sub>(a)”
4. Se determina automáticamente cuál de los tres casos del Teorema Maestro corresponde.
5. Se muestra la complejidad temporal resultante.
6. Se genera una expansión por niveles de la recurrencia.
7. Se dibuja un árbol de recurrencia representando visualmente dicha expansión.

## resultados

El panel de resultados muestra:

- Los parámetros detectados.
- El caso del Teorema Maestro encontrado.
- El valor de log<sub>b</sub>(a).
- La complejidad temporal resultante.
- La expansión por niveles.

## Árbol de recurrencia

Debajo de los resultados se dibuja un árbol de recurrencia.

Cada nodo representa un subproblema generado durante la expansión.

Cuando la cantidad de nodos crece demasiado, el sistema deja de dibujarlos individualmente para mantener el gráfico legible y muestra el patrón general:

Patrón detectado:

Nivel  $i \rightarrow a^i$  problemas de tamaño  $n/b^i$

Esto evita que el árbol ocupe un espacio excesivo en pantalla.

## Desbloqueo de la ciudad

Para completar la Ciudad 10 el usuario debe descubrir los tres casos del Teorema Maestro:

- Caso 1
- Caso 2
- Caso 3

Cada vez que aparece un caso nuevo, el sistema lo registra automáticamente y lo verá por indicadores.

## **Victoria**

Cuando los tres casos han sido descubiertos, el sistema muestra un mensaje indicando que la Ciudad 10 fue completada, concretando el final del juego.

## **Manejo de errores**

Si la recurrencia ingresada no tiene un formato válido, el sistema muestra un mensaje indicando que debe intentarse nuevamente.

Si “ $a \leq 0$ ” o “ $b \leq 1$ ”, se informa el error correspondiente y el análisis no se realiza.

Además, cuando el árbol de recurrencia supera una cantidad razonable de nodos, el sistema reemplaza el dibujo completo por una representación resumida para evitar problemas de visualización.

//como usar el programa

# **Manual del Programador**

## **Menú Principal y Mapa General (AI-Quest)**

### **Objetivo**

El package principal contiene los componentes encargados de iniciar la aplicación, administrar las partidas guardadas, gestionar el progreso del jugador y coordinar la navegación entre las distintas ciudades del juego.

---

## **Main**

### **Responsabilidad**

Es el punto de entrada de la aplicación.

## Funcionamiento

Al ejecutarse:

1. Inicializa la interfaz gráfica utilizando `SwingUtilities.invokeLater()`.
2. Crea una instancia de `VentanaMenuPrincipal`.
3. Muestra el menú principal del juego.

## Método principal

`main(String[] args)`

Inicia toda la aplicación y delega la creación de ventanas al hilo gráfico de Swing.

---

## VentanaMenuPrincipal

### Responsabilidad

Gestionar las operaciones iniciales del juego:

- Crear una nueva partida.
- Cargar una partida existente.
- Borrar partidas guardadas.
- Salir del sistema.

### Atributos

`ProgresoJuego progreso`

Almacena la partida actualmente cargada o creada.

---

### Métodos principales

`iniciarNuevaPartida()`

Solicita el nombre del jugador.

Luego:

1. Crea un objeto `ProgresoJuego`.
2. Guarda el nombre.

3. Persiste la partida.
  4. Abre el mapa principal.
- 

### **cargarPartida()**

Carga una partida previamente guardada.

Proceso:

1. Obtiene la lista de partidas disponibles.
  2. Permite seleccionar una.
  3. Recupera el progreso desde disco.
  4. Abre el mapa.
- 

### **borrarPartida()**

Elimina una partida almacenada.

Proceso:

1. Lista las partidas existentes.
  2. Permite seleccionar una.
  3. Borra el archivo correspondiente.
- 

### **abrirMapa()**

Crea la ventana principal del mapa.

Configuraciones:

- Tamaño 1000x700.
  - Crea un **PanelMapa**.
  - Asocia el JFrame al panel.
  - Reemplaza la ventana actual.
- 

## **PanelMapa**

### **Responsabilidad**

Representar visualmente el mapa del juego y detectar la selección de ciudades.

---

## Atributos

### Image mapa

Imagen de fondo del mapa.

---

### JFrame frame

Ventana contenedora utilizada para cambiar paneles dinámicamente.

---

### ProgresoJuego progreso

Referencia al progreso actual del jugador.

Permite verificar qué ciudades están desbloqueadas.

---

## Métodos principales

### paintComponent(Graphics g)

Dibuja la imagen del mapa escalada al tamaño actual de la ventana.

---

### eventos()

Registra el listener de mouse.

Detecta:

- Coordenada X.
- Coordenada Y.

Y llama a:

verificarCiudad(x, y);

---

### setFrame(JFrame frame)

Asocia el JFrame que contiene el mapa.

Es necesario para permitir cambios dinámicos de paneles.

---

### **cambiarPanel(JPanel panel)**

Reemplaza el contenido actual del JFrame.

Proceso:

```
frame.setContentPane(panel);  
frame.revalidate();  
frame.repaint();
```

---

### **verificarCiudad(int x, int y)**

Es el núcleo de navegación del mapa.

Responsabilidades:

- Detectar qué ciudad fue seleccionada.
- Verificar si está desbloqueada.
- Abrir la ciudad correspondiente.

Cada ciudad posee una región rectangular definida mediante coordenadas.

Ejemplo:

```
if (x >= 698 && x <= 911  
    && y >= 109 && y <= 244)
```

corresponde a la Ciudad 8.

---

## **ProgresoJuego**

### **Responsabilidad**

Administrar el estado completo de una partida.

Incluye:

- Nombre del jugador.
- Ciudades desbloqueadas.



- Persistencia en disco.

Implementa:

Serializable

para permitir guardado mediante serialización.

---

## Atributos

### **String nombreJugador**

Nombre asociado a la partida.

---

### **boolean[] ciudades**

Vector que almacena el estado de desbloqueo.

Ejemplo:

```
ciudades[4] == true
```

indica que la Ciudad 4 está disponible.

---

## Constructor

Inicializa:

```
ciudades = new boolean[11];
```

y habilita las ciudades iniciales:

```
ciudades[1] = true;
```

```
ciudades[2] = true;
```

---

## Métodos principales

### **desbloquear(int ciudad)**

Marca una ciudad como disponible.

```
ciudades[ciudad] = true;
```

---

### **estaDesbloqueada(int ciudad)**

Devuelve:

true

si la ciudad está habilitada.

---

### **setNombreJugador(String nombreJugador)**

Asigna el nombre de la partida.

---

### **getNombreJugador()**

Obtiene el nombre del jugador.

---

### **guardar()**

Serializa el objeto completo.

Archivo generado:

partidas/nombreJugador.dat

Proceso:

1. Crea la carpeta si no existe.
  2. Abre un `ObjectOutputStream`.
  3. Escribe el objeto actual.
  4. Cierra el flujo.
- 

### **cargar(String nombre)**

Método estático.

Recupera una partida desde disco.

Proceso:

1. Abre el archivo.
  2. Lee el objeto serializado.
  3. Devuelve un **ProgresoJuego**.
- 

## **listarPartidas()**

Método estático.

Obtiene todos los archivos:

\*.dat

de la carpeta:

partidas

y devuelve sus nombres.

---

## **borrar(String nombre)**

Método estático.

Elimina físicamente:

partidas/nombre.dat

---

# **VentanaPrincipal**

## **Responsabilidad**

Actuar como contenedor principal alternativo para el mapa.

Aunque actualmente gran parte de la navegación se realiza desde **VentanaMenuPrincipal**, esta clase encapsula la creación directa del mapa dentro de un JFrame.

---

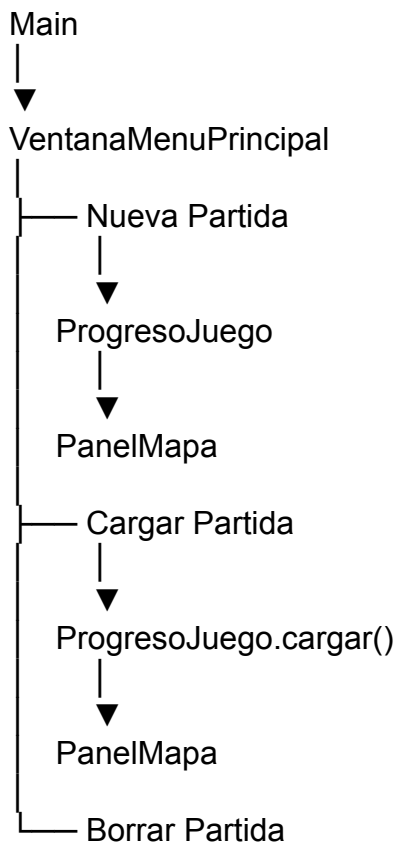
## **Constructor**

**VentanaPrincipal(ProgresoJuego progreso)**

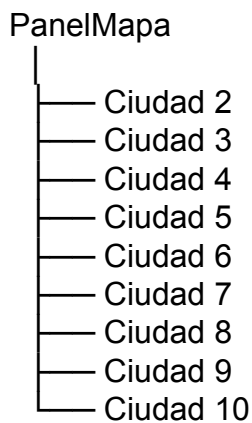
Realiza:

1. Creación de la ventana principal.
  2. Instanciación del mapa.
  3. Asociación del JFrame al mapa.
  4. Visualización inmediata.
- 

## Flujo General del Sistema



Dentro del mapa:



Cada ciudad utiliza el objeto **ProgresoJuego** compartido para consultar y actualizar el avance del jugador, permitiendo que el desbloqueo de ciudades se conserve entre ejecuciones del juego mediante serialización.

## Ciudad 1: Recolección de Matriz

### 1. Clase **Partida**

#### Descripción

Representa el estado general de una partida

#### Atributos

- **private Jugador jugador**

Jugador que recorre el laberinto y recolecta objetos.

- **private Tablero tablero**

Mapa tridimensional donde se desarrolla la partida.

- **private String mensaje**

Mensaje mostrado al usuario en la interfaz.

- **private GeneradorTablero generador**

.Generador encargado de construir el tablero

- **private int elementosRecolectados**

Cantidad de objetos importantes encontrados.

- **private boolean victoria**

Indica si el jugador ya ganó la partida.

- **private boolean partidaTerminada**

Indica si la partida finalizó.

- **private ProgresoJuego progreso**

Referencia al sistema de progreso general del juego. Permite desbloquear nuevas ciudades y guardar el avance del jugador.

- **private Runnable accionVictoria**

Acción que se ejecuta automáticamente cuando el jugador completa la Ciudad 1.

## Métodos

- **public Partida()**

Inicializa una nueva partida con tablero, jugador y objetos.

- **public void abrirCofre()**

Abre el cofre de la posición actual y aplica su efecto correspondiente.

- **public boolean moverJugador(int dx, int dy, int dz)**

Mueve al jugador si la posición destino es válida.

- **public void avanzarTurno()**

Actualiza los efectos temporales activos del jugador.

- **public boolean subirPiso()**

Permite subir un piso utilizando una escalera.

- **public boolean bajarPiso()**

Permite bajar un piso utilizando una escalera.

- **public void interactuar()**

Ejecuta la acción correspondiente al casillero actual.

- **public void usarElemento(int index)**

Utiliza un elemento almacenado en la mochila.

- **public GeneradorTablero getGenerador()**

Devuelve el generador utilizado por la partida.

- **public Jugador getJugador()**

Devuelve el jugador actual.

- **public Tablero getTablero()**

Devuelve el tablero de la partida.

- **public int getElementosRecolectados()**

Devuelve la cantidad de objetos recolectados.

- **public String getMensaje()**

Devuelve el mensaje mostrado en pantalla.

- **public void setMensaje(String mensaje)**

Actualiza el mensaje mostrado al jugador.

- **private boolean verificarVictoria()**

Comprueba si el jugador reunió los tres elementos requeridos. En caso afirmativo, marca la partida como terminada, desbloquea la Ciudad 2, guarda el progreso, muestra un mensaje de felicitación y vuelve al mapa.

## 2. Clase **Jugador**

### Descripción

Representa al personaje controlado por el usuario. Almacena su posición, visión, mochila y efectos temporales.

### Atributos

- **private static final int POSICION\_INICIAL**

Posición inicial utilizada al crear el jugador.

- **private static final int VISION\_INICIAL**

Cantidad inicial de visión del jugador.

- **private int posX**

Coordenada X actual.

- **private int posY**

Coordenada Y actual.

- **private int posZ**  
Piso actual del jugador.
- **private int visionActual**  
Radio de visión actual.
- **private int visionMaxima**  
Máximo alcance de visión disponible.
- **private int turnosInterferencia**  
Cantidad de turnos restantes bajo interferencia.
- **private Mochila mochila**  
Inventario donde se almacenan los objetos.
- **private Partida partida**  
Referencia a la partida actual.

## Métodos

- **public Jugador()**  
Crea un jugador con posición y visión iniciales.
- **public void mover(int dx, int dy, int dz)**  
Modifica la posición del jugador.
- **public void mejorarVision()**  
Aumenta permanentemente el alcance de visión.
- **public void aplicarInterferencia(int turnos)**  
Reduce temporalmente la visión del jugador.
- **public void agregarElemento(Elemento elemento)**  
Agrega un objeto a la mochila.
- **public void actualizarEfectos()**



Actualiza los efectos temporales activos.

- **public int getPosX()**

Devuelve la coordenada X actual.

- **public int getPosY()**

Devuelve la coordenada Y actual.

- **public int getPosZ()**

Devuelve el piso actual.

- **public int getVision()**

Devuelve la visión actual.

- **public Mochila getMochila()**

Devuelve la mochila del jugador.

- **public Partida getPartida()**

Devuelve la partida asociada.

- **public void setPosicion(int x, int y, int z)**

Actualiza la posición del jugador.

- **public void setPartida(Partida partida)**

Asocia el jugador a una partida.

- **public String toString()**

Devuelve una representación textual del jugador.

### 3. Clase **Mochila**

#### Descripción

Administra los objetos recolectados por el jugador. Permite almacenar, consultar y verificar elementos dentro del inventario.

#### Atributos

- **private static final int MAX\_ELEMENTOS**

Cantidad máxima de objetos que puede almacenar la mochila.

- **private ArrayList<Elemento> elementos**

Lista de elementos guardados por el jugador.

## Métodos

- **public Mochila()**

Crea una mochila vacía.

- **public void agregarElemento(Elemento elemento)**

Agrega un elemento a la mochila si hay espacio disponible.

- **public boolean posicionValida(int posicion)**

Verifica si una posición corresponde a un elemento existente.

- **public int cantidadElementos()**

Devuelve la cantidad de elementos almacenados.

- **public Elemento getElemento(int posicion)**

Devuelve el elemento ubicado en la posición indicada.

- **public String getNombreElemento(int posicion)**

Devuelve el nombre del elemento almacenado en la posición indicada.

- **public ArrayList<Elemento> getElementos()**

Devuelve una copia de la lista de elementos.

- **public boolean contieneElemento(Class<?> tipo)**

Verifica si la mochila contiene un elemento de un tipo determinado.

- **public boolean estaLlena()**

Indica si la mochila alcanzó su capacidad máxima.

- **public String toString()**

Devuelve una representación textual de la mochila.

---

## 4. Clase **Tablero**

### Descripción

Representa el mapa tridimensional del juego mediante una estructura de vectores que contiene todos los casilleros.

### Atributos

- **private static final int ANCHO**  
Cantidad de columnas del tablero.
- **private static final int ALTO**  
Cantidad de filas del tablero.
- **private static final int PISOS**  
Cantidad de niveles que posee el mapa.
- **private Vector<Vector<Vector<Casillero>>> mapa**  
Estructura tridimensional que almacena los casilleros del tablero.

### Métodos

- **public Tablero()**  
Construye un tablero vacío e inicializa todos sus casilleros.
- **public Casillero getCasillero(int x, int y, int z)**  
Devuelve el casillero ubicado en la posición indicada.
- **public boolean posicionValida(int x, int y, int z)**  
Verifica si una posición pertenece al tablero.
- **public int getAncho()**  
Devuelve el ancho del tablero.

- **public int getAlto()**

Devuelve la altura del tablero.

- **public int getPisos()**

Devuelve la cantidad de pisos del mapa.

---

## 5. Clase Casillero

### Descripción

Representa una posición individual dentro del tablero. Puede contener paredes, cofres, escaleras o espacios vacíos.

### Atributos

- **private TipoCasillero tipo**

Tipo de contenido presente en el casillero.

- **private Cofre cofre**

Cofre asociado al casillero, si existe.

### Métodos

- **public Casillero()**

Crea un casillero vacío.

- **public TipoCasillero getTipo()**

Devuelve el tipo de casillero.

- **public Cofre getCofre()**

Devuelve el cofre asociado.

- **public void setTipo(TipoCasillero tipo)**

Modifica el tipo del casillero.

- **public void setCofre(Cofre cofre)**

Asocia un cofre al casillero.

- **public boolean tieneCofre()**

Indica si el casillero contiene un cofre.

- **public boolean esPared()**

Verifica si el casillero es una pared.

- **public boolean esEscaleraSube()**

Verifica si el casillero contiene una escalera de subida.

- **public boolean esEscaleraBaja()**

Verifica si el casillero contiene una escalera de bajada.

- **public boolean estaVacio()**

Verifica si el casillero no contiene elementos especiales.

- **public void interactuar(Jugador jugador, Partida partida)**

Ejecuta la acción correspondiente según el contenido del casillero.

---

## 6. Clase **Cofre**

### Descripción

Representa un cofre del tablero que puede contener objetos útiles, trampas o estar vacío.

### Atributos

- **private Elemento contenido**

Elemento almacenado dentro del cofre.

- **private boolean abierto**

Indica si el cofre ya fue abierto.

### Métodos

- **public Cofre(Elemento contenido)**

Crea un cofre con el contenido especificado.

- **public Elemento getContenido()**

Devuelve el contenido almacenado.

- **public boolean estaAbierto()**

Indica si el cofre ya fue abierto.

- **public boolean tieneContenido()**

Verifica si el cofre contiene un elemento.

- **public void abrir()**

Marca el cofre como abierto.

- **public void vaciar()**

Elimina el contenido del cofre.

- **public String toString()**

Devuelve una descripción textual del cofre.

---

## 7. Enum **TipoCasillero**

### Descripción

Enumerado que define todos los tipos de casilleros posibles dentro del tablero.

### Constantes

- **VACIO**

Representa una casilla libre.

- **PARED**

Representa una pared que bloquea el movimiento.

- **ESCALERA\_SUBE**

Representa una escalera que permite subir de piso.

- **ESCALERA\_BAJA**

Representa una escalera que permite bajar de piso.

- **COFRE**

Representa una casilla que contiene un cofre.

## 8. Clase **Elemento**

### Descripción

Clase abstracta que representa cualquier elemento presente dentro de un cofre. Sirve como base para objetos utilizables y trampas.

### Atributos

- **protected String nombre**

Nombre identificador del elemento.

- **protected String descripcion**

Descripción breve de la función del elemento.

### Métodos

- **public Elemento()**

Constructor base de la clase.

- **public String getNombre()**

Devuelve el nombre del elemento.

- **public String getDescripcion()**

Devuelve la descripción del elemento.

- **public abstract void usar(Jugador jugador)**

Ejecuta el efecto del elemento sobre el jugador.

- **public String toString()**

Devuelve el nombre del elemento en formato texto.

---

## 9. Clase **ElementoUtilizable**

### Descripción

Clase abstracta derivada de Elemento. Representa objetos que el jugador puede guardar en la mochila y utilizar posteriormente.

### Métodos

- **public ElementoUtilizable()**

Constructor base para elementos utilizables.

- **public abstract void usar(Jugador jugador)**

Ejecuta el efecto específico del elemento.

---

## 10. Clase **Trampa**

### Descripción

Clase abstracta derivada de Elemento. Representa trampas que se activan automáticamente al abrir ciertos cofres.

### Métodos

- **public Trampa()**

Constructor base para trampas.

- **public abstract void usar(Jugador jugador)**

Aplica el efecto negativo correspondiente sobre el jugador.

---

## 11. Clase **AntorchaPerdida**



## Descripción

Elemento utilizable que aumenta permanentemente el alcance de visión del jugador.

## Métodos

- **public AntorchaPerdida()**

Inicializa el nombre y descripción de la antorcha.

- **public void usar(Jugador jugador)**

Incrementa la visión del jugador y muestra un mensaje informativo.

---

## 12. Clase MapaRasgado

### Descripción

Elemento utilizable que proporciona una pista sobre la ubicación del Amuleto Místico.

### Métodos

- **public MapaRasgado()**

Inicializa el nombre y descripción del objeto.

- **public void usar(Jugador jugador)**

Muestra una pista relacionada con la ubicación del objeto importante.

---

## 13. Clase AmuletoMistico

### Descripción

Elemento utilizable que revela las coordenadas actuales del jugador dentro del tablero.

### Métodos

- **public AmuletoMistico()**

Inicializa el nombre y descripción del amuleto.

- **public void usar(Jugador jugador)**

Muestra la posición actual del jugador mediante un mensaje.

---

## 14. Clase **Interferencia**

### Descripción

Trampa que reduce temporalmente el alcance de visión del jugador.

### Métodos

- **public Interferencia()**

Inicializa el nombre y descripción de la trampa.

- **public void usar(Jugador jugador)**

Aplica un efecto de interferencia durante varios turnos.

---

## 15. Clase **PortalInestable**

### Descripción

Trampa que teletransporta al jugador al punto inicial del mapa.

### Métodos

- **public PortalInestable()**

Inicializa el nombre y descripción de la trampa.

- **public void usar(Jugador jugador)**

Reubica al jugador en la posición inicial del juego.

---

## 16. Clase **GeneradorTablero**

### Descripción

Se encarga de construir el mapa tridimensional de la Ciudad 1. Genera paredes, escaleras, cofres, trampas y objetos utilizando distintas variantes de diseño.

## Atributos

- **private int variantePiso2**

Variante seleccionada para el tercer piso del tablero.

- **private int amuletoX**

Coordenada X donde se encuentra el Amuleto Místico.

- **private int amuletoY**

Coordenada Y donde se encuentra el Amuleto Místico.

## Métodos

- **public void generar(Tablero tablero)**

Genera completamente el contenido del tablero.

- **private void colocarAmuleto(Tablero tablero, int x, int y)**

Ubica el Amuleto Místico dentro de un cofre.

- **public int getVariantePiso2()**

Devuelve la variante utilizada para el piso superior.

- **private void construirParedesExternas(Tablero tablero, int z)**

Construye los límites exteriores de un piso.

- **private void colocarEscalera(Tablero tablero, int x, int y, int zBajo, int zAlto)**

Conecta dos pisos mediante una escalera.

- **private void colocarCofre(Tablero tablero, int x, int y, int z, Elemento contenido)**

Crea un cofre en la posición indicada.

- **private void construirPiso0(Tablero tablero, int variante)**

Genera la distribución del piso 0.

- **private void construirPiso1(Tablero tablero, int variante)**

Genera la distribución del piso 1.

- **private void construirPiso2(Tablero tablero, int variante)**

Genera la distribución del piso 2.

- **private void construirPiso0A/B/C(...)**

Generan las distintas configuraciones posibles del primer piso.

- **private void construirPiso1A/B/C(...)**

Generan las distintas configuraciones posibles del segundo piso.

- **private void construirPiso2A/B/C()**

Generan las distintas configuraciones posibles del tercer piso.

- **public int getAmuletoX()**

Devuelve la coordenada X del Amuleto Místico.

- **public int getAmuletoY()**

Devuelve la coordenada Y del Amuleto Místico.

- **private void setSeguro()**

Modifica un casillero evitando sobrescribir cofres o escaleras.

- **private void lineaParedH()**

Construye una línea horizontal de paredes.

- **private void lineaParedV()**

Construye una línea vertical de paredes.

- **private void rectanguloPared()**

Construye una región rectangular de paredes.

## 17. Clase **Sprites**

### Descripción

Carga y almacena todas las imágenes utilizadas por la Ciudad 1. Centraliza el acceso a los sprites del mapa, jugador, objetos e interfaz.

### Atributos

- **public final BufferedImage suelo**  
Imagen utilizada para representar el suelo.
- **public final BufferedImage pared**  
Imagen utilizada para representar las paredes.
- **public final BufferedImage cofre**  
Imagen utilizada para representar los cofres.
- **public final BufferedImage escaleraSubir**  
Imagen utilizada para las escaleras de subida.
- **public final BufferedImage escaleraBajar**  
Imagen utilizada para las escaleras de bajada.
- **public final BufferedImage jugador**  
Imagen utilizada para representar al jugador.
- **public final BufferedImage slotVacio**  
Imagen utilizada para los espacios vacíos del inventario.
- **public final BufferedImage antorcha**  
Imagen asociada a la Antorcha Perdida.
- **public final BufferedImage mapaRasgado**

Imagen asociada al Mapa Rasgado.

- **public final BufferedImage amuletoMistico**

Imagen asociada al Amuleto Místico.

## Métodos

- **public Sprites()**

Carga todas las imágenes necesarias para el juego.

- **private BufferedImage cargarImagen(String ruta)**

Lee una imagen desde el disco y la devuelve como BufferedImage.

---

## 18. Clase **RenderizadorBitmap**

### Descripción

Se encarga de dibujar gráficamente el estado actual de la partida utilizando bitmaps. Renderiza el mapa, el jugador, los objetos y la interfaz de usuario.

### Atributos

- **private static final int TAM\_CASILLA**

Tamaño en píxeles de cada casilla del mapa.

- **private static final int OFFSET\_X**

Desplazamiento horizontal donde comienza el mapa.

- **private static final int OFFSET\_Y**

Desplazamiento vertical donde comienza el mapa.

- **private static final int SLOT\_SIZE**

Tamaño de los espacios del inventario.

- **private static final int ITEM\_SIZE**

Tamaño de los íconos de los objetos.

- **private final Sprites sprites**

Contenedor con todas las imágenes utilizadas durante el renderizado.

## Métodos

- **public RenderizadorBitmap()**

Inicializa el renderizador y carga los sprites.

- **public Bitmap renderizar(Partida partida)**

Genera una imagen completa del estado actual de la partida.

- **private void dibujarMapa(Bitmap bitmap, Tablero tablero, Jugador jugador)**

Dibuja el mapa visible y la posición del jugador.

- **private void dibujarImagen(Bitmap bitmap, BufferedImage imagen, int x, int y)**

Copia una imagen sobre el bitmap principal.

- **private void dibujarHUD(Bitmap bitmap, Partida partida)**

Dibuja la interfaz de usuario con mensajes e inventario.

- **private void dibujarSlot(Bitmap bitmap, int x, int y, Mochila mochila, int posicion, String tecla)**

Dibuja un espacio del inventario y su contenido.

- **private BufferedImage getImagenItem(String nombre)**

Devuelve la imagen asociada a un objeto según su nombre.

---

## 19. Clase PanelCiudad1

### Descripción

Panel principal de la Ciudad 1. Gestiona la entrada del teclado, la interacción con la partida y la actualización de la interfaz gráfica.

## Atributos

- **private JLabel imagen**  
Componente utilizado para mostrar el bitmap renderizado.
- **private Partida partida**  
Partida actualmente en ejecución.
- **private RenderizadorBitmap renderizador**  
Objeto encargado de generar la representación gráfica del juego.

## Métodos

- **public PanelCiudad1(Partida partida)**  
Inicializa el panel y configura la entrada por teclado.
  - **public void actualizar()**  
Actualiza la imagen mostrada según el estado actual de la partida.
  - **public void keyPressed(KeyEvent evento)**  
Procesa las teclas presionadas por el jugador.
  - **public void keyReleased(KeyEvent evento)**  
Método requerido por KeyListener. No realiza acciones.
  - **public void keyTyped(KeyEvent evento)**  
Método requerido por KeyListener. No realiza acciones.
- 

## 20. Clase **VentanaCiudad1**

### Descripción

Ventana principal de la Ciudad 1. Crea la partida, la conecta con el sistema de progreso del juego y configura el cierre automático al completar la ciudad.

### atributo



- **private ProgresoJuego progreso**

Referencia al sistema de progreso compartido entre las distintas ciudades.

## Métodos

- **public VentanaCiudad1()**

Crea una nueva partida, configura la ventana y muestra el panel principal.

## Ciudad 2: Problema de las N Reinas

### 1. Clase **TableroReinas**

#### Descripción

Representa el tablero de  $N \times N$  donde se ubican las reinas. Guarda, para cada fila, la columna en la que hay una reina (o VACIO si no hay ninguna) y sabe decidir si una posición es segura.

#### Atributos

- **public static final int VACIO**

Constante que indica que una fila no tiene reina (valor -1).

- **private int cantidadReinas**

Cantidad de reinas, que es también el tamaño  $N$  del tablero.

- **private int[] posiciones**

Arreglo donde `posiciones[fila]` es la columna de la reina de esa fila.

#### Métodos

- **public TableroReinas(int cantidadReinas)**

Crea un tablero  $N \times N$  vacío.

- **public TableroReinas(TableroReinas otro)**

Constructor copia: crea un tablero nuevo con el mismo estado que otro.

- **public void limpiarTablero()**

Deja todas las filas sin reina (en VACIO).

- **public void colocarReina(int fila, int columna)**

Coloca una reina en la fila y columna indicadas.

- **public void quitarReina(int fila)**

Quita la reina de la fila indicada.

- **public boolean hayReina(int fila, int columna)**

Devuelve true si hay una reina en esa celda.

- **public boolean esPosicionSegura(int fila, int columna)**

Devuelve true si colocar una reina ahí no es atacada por ninguna otra ya colocada, ni por columna ni por diagonal.

- **public int getColumnaReina(int fila)**

Devuelve la columna de la reina en esa fila, o VACIO.

- **public int getCantidadReinas()**

Devuelve la cantidad de reinas (tamaño N).

- **public int[] getPosiciones()**

Devuelve una copia del vector de posiciones.

- **public String toString()**

Devuelve una representación en texto del tablero (Q = reina, . = vacío).

## 2. Clase **BacktrackingReinas**

### Descripción

Resuelve el problema de las N reinas mediante backtracking a partir de una reina inicial ya colocada. Registra cada colocación y cada retroceso para poder visualizar el proceso paso a paso.

### Atributos

- **private TableroReinas tablero**

Tablero sobre el que se resuelve el problema.

- **private List<int[]> pasos**

Lista con una copia del estado del tablero en cada paso.

- **private boolean solucionEncontrada**

Indica si la última resolución encontró solución.

## Métodos

- **public BacktrackingReinas(TableroReinas tablero)**

Inicializa el solver con el tablero recibido y sin pasos registrados.

- **public boolean resolver(int filaInicial)**

Coloca las reinas restantes en las filas libres usando backtracking, guardando los pasos. Devuelve true si encontró solución.

- **private boolean resolverDesde(List<Integer> filasLibres, int indiceFilas, int reinasColocadas, int totalReinas)**

Método recursivo: prueba colocar la reina de la fila libre actual en cada columna segura y sigue; si no llega a solución, retrocede.

- **private void guardarPaso()**

Agrega a la lista de pasos una copia del estado actual del tablero.

- **public List<int[]> getPasos()**

Devuelve la lista de pasos registrados.

- **public boolean isSolucionEncontrada()**

Devuelve true si la última llamada a resolver encontró solución.

- **public TableroReinas getTablero()**

Devuelve el tablero con la solución final.

- **public int getCantidadPasos()**

Devuelve cuántos pasos se registraron.

## 3. Clase **Interfaz**

### Descripción

Se encarga de dibujar el tablero de cada paso y guardarlo como imagen BMP. Marca en rojo la reina retirada cuando el paso es un retroceso.

### Atributos

- **private static final Color COLOR\_CELDA\_CLARA / COLOR\_CELDA\_OSCURA**

Colores de las celdas claras y oscuras del tablero.

- **private static final Color COLOR\_REINA / COLOR\_BACKTRACK / COLOR\_FONDO**

Color de la reina (dorado), del retroceso (rojo) y del fondo.

- **private String carpetaSalida**

Carpeta donde se guardan las imágenes generadas.

## Métodos

- **public Interfaz(String carpetaSalida)**

Inicializa el renderizador y crea la carpeta de salida si no existe.

- **public void generarPasos(List<int[]> pasos, int cantidadReinas, int tamTotal)**

Genera una imagen BMP por cada paso, dibujando el tablero al tamaño indicado.

- **private BufferedImage dibujarTablero(int[] posiciones, int n, boolean esBacktrack, int numeroPaso, int tamTotal)**

Devuelve la imagen del tablero con las reinas en sus posiciones, marcando en rojo la última reina si el paso fue un retroceso.

- **private void dibujarReina(Graphics2D g, int x, int y, int tamCelda, boolean esBacktrack)**

Dibuja una reina en la celda, dorada o roja según corresponda.

- **private void guardarImagen(BufferedImage imagen, int numeroPaso)**

Guarda la imagen con el nombre paso\_XXXX.bmp.

- **private boolean esRetroceso(int[] posAnterior, int[] posActual, int n)**

Devuelve true si entre el paso anterior y el actual se quitó una reina.

- **private int contarReinas(int[] posiciones, int n)**

Devuelve cuántas reinas hay colocadas en el vector.

- **private int ultimaFilaConReina(int[] posiciones, int n)**

Devuelve la última fila que tiene una reina, o -1.

#### 4. Clase **PanelCiudad2**

##### Descripción

Panel de Swing que arma toda la interfaz de la Ciudad 2. Tiene dos pantallas (presentación y juego) mediante un CardLayout, lee los parámetros, lanza el backtracking, reproduce el paso a paso y verifica la victoria.

##### Atributos

- **private JLabel labelImagen / labelInfo**

Muestran la imagen del paso actual y el cartel de resultado.

- **private JSlider sliderPasos**

Permite recorrer los pasos manualmente.

- **private JButton btnAnterior / btnSiguiente / btnResolver / btnSalirCiudades**

Botones de navegación, de resolver y de salir al mapa.

- **private JSpinner spinnerReinas / spinnerFila / spinnerColumna**

Selectores de cantidad de reinas y posición inicial.

- **private int pasoActual / totalPasos**

Paso que se muestra y cantidad total de pasos.

- **private boolean ganada**

Indica si la ciudad ya fue ganada.

- **private Runnable accionSalir**

Acción a ejecutar al salir a las ciudades.

##### Métodos

- **public PanelCiudad2()**

Inicializa el panel con las dos pantallas y muestra la de presentación.

- **private JPanel construirPanelPresentacion() / construirPanelJuego()**

Arman la pantalla de presentación y la de juego.

- **private void mostrarJuego()**

Cambia de la pantalla de presentación a la de juego.

- **private void lanzarResolucion()**

Lee los parámetros, ejecuta el backtracking, genera las imágenes, habilita la navegación y verifica la victoria.

- **private void verificarVictoria(boolean encontro)**

Si se resolvió en menos de 10 pasos, marca la ciudad como ganada y habilita el botón de salir.

- **public void setAccionSalir(Runnable accion)**

Define la acción de salir a las ciudades.

- **private void mostrarPaso(int numeroPaso) / irAPaso(int numeroPaso)**

Muestran y navegan a un paso dentro del rango válido.

- **private void actualizarRangoPosicionInicial()**

Ajusta el máximo de los spinners de fila y columna a la cantidad de reinas elegida.

## Ciudad 3: Laberinto Hacker

### 1. Clase SalidaLaberinto

**Descripción** Clase principal de la Ciudad 3. Controla la lógica del algoritmo de Backtracking (DFS) adaptado para moverse en cuatro direcciones, el refresco de la interfaz gráfica del laberinto, el manejo de hilos (Threads) para la animación y la persistencia de datos (desbloqueo de nivel).

### Atributos

- **private static ProgresoJuego progreso** Referencia al progreso global para desbloquear la siguiente ciudad.
- **static boolean hackeoEnProgreso / static boolean abortarHackeo** Banderas de control de estado para bloquear botones durante la animación o interrumpir la recursión manualmente.

- `static int MARGEN_X / static int MARGEN_Y` Variables dinámicas que se recalculan en cada ejecución para asegurar que cualquier matriz (sea 5x5, 6x8, etc.) quede centrada en el lienzo.

## Métodos

- `public static boolean resolverLaberinto(int[][] laberinto, int x, int y, int prevX, int prevY, int[][] solucion, Bitmap bmp)` Método recursivo principal. Evalúa las 4 direcciones cardinales. Retorna `false` si es interceptado por `abortarHackeo`. Pinta y despinta la matriz `solucion` (backtracking) llamando al redibujado de la interfaz.
- `public static void repintarLaberinto(...)` Refresca el estado visual del tablero completo en base a la matriz de soluciones actuales. Previene el *flickering* (parpadeo de pantalla).
- `public static void dibujarCartelFinal(...)` / `dibujarCartelAbortado(...)` Generan por código vectorial (líneas, cuadrados, círculos) los diseños de los mensajes finales (Calavera, Escudo, Advertencia).
- `public static boolean esSeguro(...)` Valida que el próximo paso no sea pared, no se salga de los límites de la matriz y no se haya visitado ya en la iteración actual para evitar bucles infinitos.

## 2. Clase MatricesEjemplo

**Descripción** Clase utilitaria que funciona como un banco de datos estático (Data Pool). Almacena matrices hardcodeadas (niveles) de diferentes proporciones y dificultades, garantizando variedad en la ejecución sin la inestabilidad de la generación azarosa pura.

### Atributos

- `private static final int[][][] BANCO_DE_LABERINTOS` Arreglo tridimensional irregular que contiene 10 laberintos de diseño propio (algunos con solución y otros bloqueados intencionalmente).

## Métodos

- `public static int[][] obtenerLaberintoAleatorio()` Selecciona un índice al azar y devuelve una copia profunda (*Deep Copy*) del laberinto seleccionado, evitando que la referencia original sea alterada durante el juego.
- `public static int[] getDimensionesMaximas()` Recorre el banco de laberintos para encontrar la mayor cantidad de filas y columnas, permitiendo que la interfaz gráfica dimensione la ventana correctamente desde el arranque.

## 3. Clase PanelCiudad3

**Descripción** Panel de transición integrado al `JFrame` principal de la aplicación. Actúa como el puente entre el Mapa Global y el minijuego de la Ciudad 3.

### Métodos

- `public PanelCiudad3(ProgresoJuego progreso)` Constructor que configura la interfaz del menú de la ciudad. Inyecta la referencia de `ProgresoJuego` dentro de `SalidaLaberinto` antes de invocar su método `main`. Administra también el retorno al `PanelMapa` reconstruyendo el contenedor principal de la aplicación para evitar errores de referencias nulas ("Frame no inicializado").

## Ciudad 4: Visualización de Algoritmos de Ordenamiento

### 1. Clase `BubbleSort`

#### Descripción

Implementa el algoritmo de ordenamiento Bubble Sort y registra cada intercambio realizado para permitir su visualización paso a paso.

#### Métodos

- `public static Queue<int[]> ordenar(int[] vector)`
    - **Función:** Ordena el arreglo de menor a mayor utilizando Bubble Sort.
    - **Parámetros:** `vector`, arreglo de enteros a ordenar.
    - **Retorno:** Una `Queue<int[]>` con los estados intermedios del arreglo.
    - **Validación:** Lanza `IllegalArgumentException` si el vector es `null`.
- 

### 2. Clase `QuickSort`

#### Descripción

Implementa el algoritmo Quick Sort utilizando recursividad y almacena los estados intermedios del proceso.

#### Atributos

- `private static Queue<int[]> pasos`
  - Cola donde se guardan las distintas configuraciones del arreglo durante la ejecución.

#### Métodos

- `public static Queue<int[]> ordenar(int[] vector)`



- Inicializa la cola de pasos y ejecuta el algoritmo.
  - **private static void quickSort(int[] v, int inicio, int fin)**
    - Aplica recursivamente Quick Sort sobre el subarreglo indicado.
  - **private static int particion(int[] v, int inicio, int fin)**
    - Reordena los elementos respecto de un pivote y devuelve la posición final de este.
- 

### 3. Clase **EstadoOrdenamiento**

#### Descripción

Representa un estado del proceso de ordenamiento almacenando el arreglo y el algoritmo utilizado.

#### Atributos

- **private int[] datos**
  - Arreglo correspondiente al estado almacenado.
- **private String algoritmo**
  - Nombre del algoritmo asociado al estado.

#### Métodos

- **public EstadoOrdenamiento(int[] datos, String algoritmo)**
    - Constructor que inicializa los atributos y valida que no sean `null`.
  - **public int[] getDatos()**
    - Devuelve el arreglo almacenado.
  - **public String getAlgoritmo()**
    - Devuelve el nombre del algoritmo.
- 

### 4. Clase **Bitmap**

#### Descripción

Se encarga de generar la representación gráfica de los datos mediante una imagen.

#### Atributos

- **private BufferedImage imagen**
  - Imagen sobre la que se dibujan las barras.
- **private Graphics2D g**
  - Objeto gráfico utilizado para realizar los dibujos.

## Métodos

- **public Bitmap(int ancho, int alto)**
    - Crea la imagen con las dimensiones indicadas e inicializa el contexto gráfico.
  - **public BufferedImage getImage()**
    - Devuelve la imagen generada.
  - **public void limpiar()**
    - Borra el contenido de la imagen pintándola de blanco.
  - **public void dibujarBarras(int[] datos)**
    - Dibuja una barra por cada elemento del arreglo y muestra su valor numérico.
- 

## 5. Clase **PanelCiudad4**

### Descripción

Panel encargado de mostrar gráficamente el estado actual del ordenamiento.

### Atributos

- **private Bitmap bmp**
  - Objeto que realiza el dibujo de las barras.
- **private JLabel lblImagen**
  - Componente donde se visualiza la imagen generada.
- **private ProgresoJuego progreso**
  - Administra el desbloqueo y guardado del progreso del juego.

### Métodos

- **public PanelCiudad4(ProgresoJuego progreso)**
    - Constructor principal que inicializa el panel.
  - **public PanelCiudad4()**
    - Constructor alternativo que crea un nuevo **ProgresoJuego**.
  - **public void mostrarDatos(int[] datos)**
    - Actualiza la representación gráfica con un nuevo estado del arreglo y registra el progreso del jugador.
- 

## 6. Clase **VentanaOrdenamientos**

### Descripción

Ventana principal de la Ciudad 4. Gestiona la interacción con el usuario y coordina la ejecución de los algoritmos.

### Atributos

- **private JTextField txtNumeros**
  - Campo donde el usuario ingresa los números.
- **private JComboBox<String> combo**
  - Selector del algoritmo de ordenamiento.
- **private JButton btnOrdenar**
  - Inicia el proceso de ordenamiento.
- **private JButton btnSalir**
  - Cierra la ventana.
- **private PanelCiudad4 panel**
  - Panel que muestra la animación gráfica.
- **private ProgresoJuego progreso**
  - Controla el estado de desbloqueo de las ciudades.
- **private boolean usoBubbleSort**
  - Indica si Bubble Sort fue utilizado.
- **private boolean usoQuickSort**
  - Indica si Quick Sort fue utilizado.

### Métodos

- **public VentanaOrdenamientos(ProgresoJuego progreso)**
    - Constructor que configura la ventana e inicializa sus componentes.
  - **private void inicializar()**
    - Crea y organiza todos los controles gráficos y registra sus eventos.
  - **private void ordenar()**
    - Lee la entrada del usuario, ejecuta el algoritmo seleccionado y obtiene los estados intermedios.
  - **private void verificarVictoria()**
    - Comprueba si se utilizaron ambos algoritmos y, de ser así, desbloquea la Ciudad 5.
  - **private void animar(Queue<int[]> pasos)**
    - Reproduce la animación consumiendo los estados almacenados en la cola mediante un **Timer**.
- 

## Ciudad 5: Búsqueda en lista vs árbol binario

# 1. Bitmap

## Descripción

Clase encargada de generar y administrar la imagen utilizada para mostrar los resultados de las búsquedas dentro de la interfaz gráfica.

## Atributos

private BufferedImage imagen;

Almacena la imagen donde se dibuja la información.

private Graphics2D g;

Objeto gráfico utilizado para realizar los dibujos sobre la imagen.

## Métodos

**Bitmap(int ancho, int alto)**

Constructor que crea la imagen con las dimensiones indicadas e inicializa el contexto gráfico.

**getImage()**

Devuelve la imagen generada.

**limpiar()**

Pinta toda la imagen de color blanco para eliminar dibujos anteriores.

**mostrarTexto(String texto)**

Muestra un texto dentro de la imagen, distribuyendo cada línea verticalmente.

---

# 2. NodoABB

## Descripción

Representa un nodo del Árbol Binario de Búsqueda.

## Atributos

String palabra;

Palabra almacenada.

int linea;

Línea del archivo donde aparece.

int posicion;

Posición dentro de la línea.

NodoABB izquierda;

Referencia al hijo izquierdo.

NodoABB derecha;

Referencia al hijo derecho.

### **Métodos**

**NodoABB(...)**

Constructor que inicializa la información del nodo.

---

## **3. ArbolBusqueda**

### **Descripción**

Implementa un Árbol Binario de Búsqueda para almacenar y localizar palabras.

### **Atributos**

private NodoABB raiz;

Nodo raíz del árbol.

private int operaciones;

Contador de operaciones realizadas durante la búsqueda.

### **Métodos**

**insertar(String palabra, int linea, int posicion)**

Inserta una nueva palabra dentro del árbol.

**insertarRec(...)**

Método recursivo encargado de ubicar el nodo en la posición correcta.

**buscar(String palabra)**

Busca una palabra en el árbol y devuelve un objeto ResultadoBusqueda.

**buscarRec(...)**

Método recursivo que recorre el árbol comparando palabras.

---

## 4. BusquedaLineal

### Descripción

Implementa una búsqueda secuencial utilizando listas.

### Atributos

private ArrayList<String> palabras;

Lista de palabras.

private ArrayList<Integer> lineas;

Lista de líneas correspondientes.

private ArrayList<Integer> posiciones;

Lista de posiciones correspondientes.

### Métodos

**BusquedaLineal()**

Inicializa las listas.

**agregar(...)**

Agrega una palabra y su ubicación.

**buscar(String palabra)**

Recorre secuencialmente todas las palabras hasta encontrar la solicitada.

---

## 5. ResultadoBusqueda

### Descripción

Clase utilizada para encapsular los resultados obtenidos por un algoritmo de búsqueda.

### Atributos

private int linea;

Línea donde se encontró la palabra.

private int posicion;

Posición de la palabra.

private long tiempo;

Tiempo de búsqueda en nanosegundos.

private int operaciones;

Cantidad de comparaciones realizadas.

### Métodos

#### Constructor

Inicializa todos los datos del resultado.

#### Getters

getLinea()

getPosicion()

getTiempo()

getOperaciones()

Devuelven los valores almacenados.

---

## 6. LectorArchivo

### Descripción

Clase encargada de leer un archivo de texto y cargar su contenido en las estructuras de búsqueda.

### Métodos

**cargarArchivo(String ruta, ArbolBusqueda arbol, BusquedaLineal lista)**

Realiza las siguientes tareas:

1. Abre el archivo.
2. Lee línea por línea.
3. Separa las palabras.

4. Convierte cada palabra a minúsculas.
  5. Inserta la información tanto en el ABB como en la estructura lineal.
- 

## 7. PanelCiudad5

### Descripción

Panel gráfico encargado de mostrar los resultados de las búsquedas.

### Atributos

private Bitmap bmp;

Objeto gráfico utilizado para dibujar.

private JLabel lblImagen;

Componente visual donde se muestra la imagen.

private ProgresoJuego progreso;

Referencia al progreso del jugador.

### Métodos

**PanelCiudad5(ProgresoJuego progreso)**

Inicializa el panel y sus componentes gráficos.

**mostrarResultado(String texto)**

Muestra el resultado de una búsqueda dentro del bitmap.

Además:

progreso.desbloquear(6);  
progreso.guardar();

actualiza el progreso del juego.

---

## 8. VentanaBusquedas

### Descripción



Es la ventana principal de la Ciudad 5.

Coordina la interfaz gráfica, la carga de archivos, la ejecución de búsquedas y el desbloqueo de la siguiente ciudad.

## **Atributos**

### **Componentes gráficos**

```
private JTextField txtPalabra;  
private JButton btnCargarArchivo;  
private JButton btnBuscar;  
private JButton btnSalir;
```

Elementos de interacción con el usuario.

### **Panel gráfico**

```
private PanelCiudad5 panel;
```

Muestra los resultados.

### **Estructuras de búsqueda**

```
private ArbolBusqueda arbol;  
private BusquedaLineal lista;
```

Contienen los datos procesados.

### **Progreso**

```
private ProgresoJuego progreso;
```

Permite actualizar el avance del jugador.

### **Control de victoria**

```
private boolean archivoCargado;
```

Indica si se cargó un archivo.

```
private int cantidadBusquedas;
```

Cuenta las búsquedas realizadas.

## **Métodos**

### **VentanaBusquedas(ProgresoJuego progreso)**

Constructor principal.

### **inicializar()**

Crea y organiza todos los componentes gráficos.

### **eventos()**

Asocia los eventos a cada botón.

**cargarArchivo()**

Permite seleccionar y procesar un archivo TXT.

**buscar()**

Ejecuta las búsquedas utilizando ambas estructuras.

**mostrarResultados(...)**

Construye el texto con los resultados y comparaciones.

**verificarVictoria()**

Comprueba si el jugador:

- cargó un archivo;
- realizó al menos tres búsquedas.

Si ambas condiciones se cumplen, desbloquea la Ciudad 6.

## Ciudad 6: Hashing

### 1. Clase TablaHash

#### Descripción

Implementa la tabla hash con encadenamiento separado. Cada bucket es una lista de palabras. Provee inserción, búsqueda y una medición real de la búsqueda, devolviendo los pasos de cada operación para visualizarlos.

#### Atributos

- `private LinkedList<String>[] tabla`

Arreglo de buckets; cada bucket es una lista (cadena) de palabras.

- `private int tamaño`

Cantidad de buckets de la tabla (conviene que sea primo).

- `private int cantidad`

Cantidad de palabras distintas guardadas.

## Métodos

- `public TablaHash(int tamaño)`

Crea una tabla vacía con tamaño buckets, cada uno una lista vacía.

- `public int sumaAscii(String palabra)`

Devuelve la suma de los valores ASCII de los caracteres de la palabra.

- `public int funcionHash(String palabra)`

Devuelve el índice del bucket aplicando suma ASCII módulo tamaño.

- `public Queue<PasoHash> insertar(String palabra)`

Inserta la palabra si no estaba (clave única) y devuelve la cola de pasos: cálculo del índice, revisión de duplicados y colisión o inserción.

- `public Queue<PasoHash> buscar(String palabra)`

Devuelve la cola de pasos de la búsqueda recorriendo la cadena del bucket hasta encontrar la palabra o agotar la cadena. No modifica la tabla.

- `public Medicion medir(String palabra)`

Ejecuta la búsqueda real, sin pasos ni pausas, midiendo comparaciones y tiempo promedio (repite muchas veces y promedia).

- `public int cantidadBucketsConAlMenos(int min)`

Devuelve cuántos buckets tienen al menos min elementos.

- `public int getCantidad() / getTamaño()`

Devuelven la cantidad de palabras y la cantidad de buckets.

- `public LinkedList<String> getBucket(int indice)`

Devuelve la lista (cadena) del bucket indicado.

## 2. Clase PasoHash

### Descripción

Representa un paso de la visualización de una operación sobre la tabla. Guarda la descripción, el bucket involucrado, la posición en la cadena y el tipo de paso, que la vista usa para elegir el color del resaltado.

#### Atributos

- `public enum Tipo`

Tipos de paso: CALCULO, COMPARACION, COLISION, ENCONTRADO, NO\_ENCONTRADO, INSERTADO, YA\_EXISTE.

- `private String descripcion`

Texto que describe el paso.

- `private int bucket / posicionEnCadena`

Bucket y posición dentro de la cadena involucrados (-1 si no aplica).

- `private Tipo tipo`

Tipo de paso, usado para el color del resaltado.

#### Métodos

- `public PasoHash(String descripcion, int bucket, int posicionEnCadena, Tipo tipo)`
- Crea el paso con todos sus datos.
- `public String getDescripcion() / int getBucket() / int getPosicionEnCadena() / Tipo getTipo()`
- Getters de cada atributo.

### 3. Clase Medicion

#### Descripción

Guarda el resultado de medir una búsqueda: cantidad de comparaciones, tiempo promedio en nanosegundos y si la palabra se encontró.

#### Atributos

- `private int comparaciones`

Comparaciones hechas en la búsqueda.

- `private long nanos`

Tiempo promedio de la búsqueda en nanosegundos.

- `private boolean encontrada`

Indica si la palabra se encontró.

#### Métodos

- `public Medicion(int comparaciones, long nanos, boolean encontrada)`

Crea la medición con sus tres datos.

- `public int getComparaciones() / long getNanos() / boolean fueEncontrada()`

Getters de cada atributo.

## 4. Clase RenderizadorHash

#### Descripción

Dibuja el estado de la tabla hash y lo guarda como imagen BMP, una por cada paso de una operación, resaltando el bucket y el elemento involucrados con el color asociado al tipo de paso.

#### Atributos

- `private static final Color COLOR_FONDO / COLOR_CELDA / COLOR_BORDE`

Colores del fondo, las celdas y los bordes y el texto.

- `private static final Color COLOR_CALCULO / COLOR_COMPARACION / COLOR_COLISION / COLOR_EXITO / COLOR_ERROR`

Colores de resaltado según el tipo de paso.

- `private static final int ALTO_FILA / ANCHO_CELDA / SEPARACION / MARGEN / ALTO_ENCABEZADO`

Medidas en píxeles del dibujo.

- `private String carpetaSalida`

Carpeta donde se guardan las imágenes generadas.

### Métodos

- `public RenderizadorHash(String carpetaSalida)`

Inicializa el renderizador y crea la carpeta de salida si no existe.

- `public void generarPasos(TablaHash tabla, List<PasoHash> pasos)`

Genera una imagen BMP por cada paso, resaltando el bucket y el elemento correspondientes.

- `private BufferedImage dibujarEstado(TablaHash tabla, PasoHash paso, int numeroPaso, int totalPasos)`

Devuelve la imagen con la tabla completa, resaltando lo indicado por el paso.

- `private Color colorDe(PasoHash.Tipo tipo)`

Devuelve el color de resaltado asociado al tipo de paso.

- `private int maxLargoCadena(TablaHash tabla)`

Devuelve la cantidad de elementos del bucket con la cadena más larga.

- `private void guardarImagen(BufferedImage imagen, int numeroPaso)`

Guarda la imagen con el nombre paso\_XXXX.bmp.

## 5. Clase PanelCiudad6

### Descripción

Panel de Swing que arma la interfaz de la Ciudad 6. Permite insertar y buscar palabras, reproduce el paso a paso con imágenes y texto, actualiza el progreso y verifica la victoria. La animación corre en un hilo aparte para no congelar la ventana.

### Atributos

- `private TablaHash tabla`

Tabla hash que se visualiza.

- `private RenderizadorHash renderizador`

Genera las imágenes de cada paso.

- `private ProgresoJuego progreso`

Administra el desbloqueo y guardado del progreso.

- `private JTextField campoPalabra / JTextArea areaExplicacion / JLabel etiquetaProgreso / labelImagen`

Campo de palabra, área de explicación, etiqueta de progreso e imagen del paso.

- `private JSlider sliderPasos / JButton botonAnterior / botonSiguientePaso / botonSiguiente`

Controles de navegación de los pasos y botón para continuar.

- `private boolean animando / ganada`

Indican si hay una animación en curso y si la ciudad ya fue ganada.

- `private int totalPasos / bucketsRequeridos / minPorBucket`

Cantidad de pasos y condiciones de victoria (buckets requeridos y mínimo por bucket).

- `private Runnable accionSiguiente`

Acción a ejecutar al pasar a la siguiente ciudad.

## Métodos

- `public PanelCiudad6(int tamanoTabla, int bucketsRequeridos, int minPorBucket, ProgresoJuego progreso)`

Arma el panel, crea la tabla y define las condiciones de victoria.

- `public PanelCiudad6(ProgresoJuego progreso)`

Constructor con la configuración por defecto del juego.

- `private void ejecutar(boolean insertar)`

Inserta o busca la palabra del campo, genera las imágenes de los pasos y los reproduce.

- `private void reproducir(String titulo, List<PasoHash> pasos, boolean esInsercion, String mensajeFinal)`

Recorre los pasos mostrando uno por vez en un hilo aparte; al terminar habilita la navegación, agrega el mensaje final y, si fue inserción, actualiza el progreso.

- `private void actualizarProgreso()`

Actualiza la etiqueta y, si se alcanzó el objetivo, marca la ciudad como ganada y desbloquea la Ciudad 7.

- `public void setAccionSiguiente(Runnable accion)`

Define la acción de pasar a la siguiente ciudad.

- `private void mostrarPaso(int numeroPaso) / irAPaso(int numeroPaso)`

Muestran y navegan a un paso dentro del rango válido.

- `public boolean estaGanada()`

Devuelve true si la ciudad ya fue ganada.

## Ciudad 7: Gestor de Grafos

### 1. Clase ProgramaPrincipalGrafos

**Descripción** Controlador principal de la Ciudad 7. Gestiona la lógica de los botones de la interfaz (Swing), mantiene el estado persistente del TDA Grafo en memoria, coordina la ejecución de los algoritmos y registra el uso de herramientas para desbloquear el progreso.

#### Atributos

- `private static Grafo<String, Integer> ciudad7`
  - Instancia global del TDA Grafo que estructura la red.
- `private static List<String> mapeoNodos`
  - Diccionario que vincula el valor del nodo (String) con un índice numérico secuencial (0, 1, ..., n-1).
- `private static boolean usoAgregarNodo` (y demás booleanos)
  - Banderas lógicas que registran qué herramientas fueron utilizadas por el usuario.
- `private static ProgresoJuego progreso`



- Controla el estado de guardado y desbloqueo del juego general.

## Métodos

- `public static void main(String[] args)`
  - Configura el lienzo principal, instancia los "ActionListeners" de los menús y lanza la ventana.
- `private static void verificarVictoria()`
  - Comprueba si todas las banderas de herramientas están en `true`. De ser así, invoca el desbloqueo de la Ciudad 8 y guarda la partida.
- `private static void actualizarLienzo()`
  - Sincroniza la estructura de datos dinámica con el motor de dibujo. Aplica técnica "Clear & Redraw" reconstruyendo el espacio de dibujo para evitar artefactos visuales.
- `private static int[][] generarMatrizBase(int valorVacio)`
  - Función adaptadora (Adapter Pattern). Transforma la Lista de Adyacencia del TDA a una Matriz de Adyacencia estática consumible por el renderizador y los algoritmos.

## 2. Clase PanelCiudad7

**Descripción** Panel de integración UI. Actúa como pantalla intermedia dentro del JFrame principal del juego. Permite instanciar el gestor de grafos independiente o retornar al mapa global.

## Métodos

- `public PanelCiudad7(ProgresoJuego progreso)`
  - Constructor que configura el `GridBagLayout`. Al iniciar el programa de grafos, invoca a `ProgramaPrincipalGrafos.main()` y, en segundo plano, reemplaza el contenido del frame padre (`PanelMapa`) para lograr una transición fluida al cerrar la ventana.

# Ciudad 8: Torres de Hanoi

## 1. Bitmap

### Descripción

Clase encargada de generar y manipular la imagen utilizada para representar gráficamente las Torres de Hanoi.

### Atributos

`private BufferedImage imagen;`

Imagen donde se realizan todos los dibujos.

`private Graphics2D g;`

Contexto gráfico utilizado para pintar sobre la imagen.

## **Métodos principales**

**`Bitmap(int ancho, int alto)`**

Crea una imagen con el tamaño especificado.

**`getImage()`**

Devuelve la imagen generada.

**`getWidth()`**

Obtiene el ancho de la imagen.

**`getHeight()`**

Obtiene el alto de la imagen.

**`rellenar(Color color)`**

Pinta completamente la imagen con un color.

**`drawLine(...)`**

Dibuja una línea.

**`drawRectangle(...)`**

Dibuja el contorno de un rectángulo.

**`fillRectangle(...)`**

Dibuja un rectángulo relleno.

**`drawCircle(...)`**

Dibuja el contorno de un círculo.

**`fillCircle(...)`**

Dibuja un círculo relleno.

**drawText(...)**

Dibuja texto sobre la imagen.

---

## 2. Movimiento

### Descripción

Representa un movimiento individual dentro de la resolución de las Torres de Hanoi.

### Atributos

private int origen;

Torre desde donde se mueve el disco.

private int destino;

Torre hacia donde se mueve el disco.

### Métodos

**Movimiento(int origen, int destino)**

Constructor que crea el movimiento.

**getOrigen()**

Devuelve la torre de origen.

**getDestino()**

Devuelve la torre de destino.

---

## 3. Hanoi

### Descripción

Implementa el algoritmo recursivo de Torres de Hanoi.

Se encarga de calcular todos los movimientos necesarios para resolver el problema.

### Atributos

private List<Movimiento> movimientos;

Lista donde se almacenan los movimientos generados por el algoritmo.

### Métodos

#### Hanoi()

Inicializa la lista de movimientos.

#### resolver(int cantidadDiscos)

Calcula todos los movimientos necesarios para resolver el problema.

Retorna:

List<Movimiento>

con la secuencia completa de movimientos.

#### mover(int discos, int origen, int destino, int auxiliar)

Método recursivo que implementa el algoritmo.

Caso base:

if (discos == 1)

genera un único movimiento.

Caso recursivo:

1. Mueve  $n-1$  discos a la torre auxiliar.
2. Mueve el disco principal.
3. Mueve los  $n-1$  discos restantes a la torre destino.

---

## 4. PanelCiudad8

### Descripción

Panel principal encargado de:

- Mostrar la interfaz gráfica.
- Ejecutar la simulación.
- Dibujar las torres.
- Animar los movimientos.
- Desbloquear la siguiente ciudad.

## Atributos

private Bitmap bmp;

Motor gráfico utilizado para realizar los dibujos.

private ProgresoJuego progreso;

Controla el avance del jugador.

private JLabel lblImagen;

Componente visual que muestra la imagen generada.

---

## Constructor

### PanelCiudad8(ProgresoJuego progreso)

Inicializa:

- El bitmap.
- La interfaz gráfica.
- Los botones.
- El temporizador de refresco.

Además dibuja el menú inicial.

---

## Métodos de PanelCiudad8

**dibujarMenu()**

Muestra la pantalla inicial.

Dibuja:

- Título de la ciudad.
  - Nombre del desafío.
- 

## **iniciar()**

Método principal de ejecución.

Realiza las siguientes tareas:

1. Crea las torres.
2. Dibuja el estado inicial.
3. Ejecuta el algoritmo de Hanoi.
4. Reproduce cada movimiento.
5. Actualiza la representación gráfica.
6. Finaliza la ciudad cuando termina la simulación.

La ejecución se realiza dentro de:

`new Thread(...)`

para evitar bloquear la interfaz gráfica.

---

## **crearTorres(int discos)**

Genera la estructura de datos que representa las torres.

Utiliza:

`List<List<Integer>>`

Cada lista interna representa una torre.

Inicialmente todos los discos se colocan en la torre 0.

Ejemplo:

`[4,3,2,1]`

[]

[]

---

## **finalizarCiudad()**

Se ejecuta cuando el algoritmo termina.

Responsabilidades:

- Mostrar mensaje de felicitación.
- Desbloquear la siguiente ciudad.
- Guardar el progreso.

progreso.desbloquear(9);

progreso.guardar();

---

## **dibujarEstado(List<List<Integer>> torres)**

Representa gráficamente el estado actual de las torres.

Dibuja:

### **Postes**

bmp.drawLine(...)

### **Base**

bmp.drawLine(...)

### **Discos**

bmp.fillRect(...)

Cada disco tiene un ancho proporcional a su tamaño.

---

## Estructura de Datos Utilizada

La simulación utiliza:

List<List<Integer>>

Ejemplo:

```
[  
[4,3],  
[2],  
[1]  
]
```

Cada lista representa una torre.

Los enteros representan los discos.

- Valores grandes → discos grandes.
- Valores pequeños → discos pequeños.

---

## Flujo General de Ejecución

1. El usuario presiona **Iniciar Hanoi**.
2. Se crean las torres iniciales.
3. El algoritmo recursivo genera todos los movimientos.
4. Los movimientos se almacenan en una lista.
5. La simulación reproduce los movimientos uno por uno.
6. El panel redibuja el estado actual.
7. Se muestra el mensaje de victoria.
8. Se desbloquea la Ciudad 9.
9. Se guarda el progreso del jugador.

## Ciudad 9 – Sistema de Combate por Turnos



## 1. Objetivo

La Ciudad 9 implementa un sistema de combate por turnos donde el jugador se enfrenta a múltiples enemigos utilizando distintas acciones. El objetivo principal de esta ciudad es demostrar la utilización de las estructuras de datos requeridas por el trabajo práctico:

- Lista
- Cola
- Pila

Además, se aplica una separación entre lógica de negocio y presentación mediante una organización por paquetes.

---

## 2. Arquitectura General

La implementación se encuentra dividida en tres paquetes principales:

Ciudad9

|

├─ bitmap

├─ modelo

└─ vista

### **bitmap**

Contiene las clases encargadas del dibujo de la interfaz gráfica.

### **modelo**

Contiene toda la lógica del combate y las estructuras de datos utilizadas.

### **vista**

Contiene las ventanas y paneles Swing utilizados para la interacción con el usuario.

---

## 3. Paquete modelo

### 3.1 Clase Personaje

Representa una entidad genérica participante del combate.

#### Responsabilidades

- Almacenar nombre.
- Almacenar vida actual.
- Recibir daño.
- Recuperar vida.
- Informar si continúa con vida.

#### Atributos

```
private String nombre;
```

```
private int vida;
```

#### Métodos principales

```
recibirDanio(int danio)
```

```
curar(int cantidad)
```

```
estaVivo()
```

---

### 3.2 Clase JugadorCombate

Hereda de Personaje.

Representa al jugador controlado por el usuario.

#### Herencia

```
JugadorCombate extends Personaje
```

Actualmente no agrega comportamiento adicional pero permite diferenciar al jugador de los enemigos.

### 3.3 Clase Enemigo

Hereda de Personaje.

Representa a cada enemigo del combate.

#### Atributos

```
private int danio;
```

#### Responsabilidades

- Mantener el daño que puede infligir.
  - Participar en la cola de turnos.
- 

### 3.4 Enumerado TipoAccion

Representa las acciones disponibles durante el combate.

#### Valores

ATAQUE

DEFENSA

HABILIDAD

---

### 3.5 Clase Accion

Representa una acción realizada por el jugador.

#### Atributos

```
private TipoAccion tipo;
```

```
private int valor;
```

#### Responsabilidades

Permitir almacenar acciones dentro de la pila de acciones para ser ejecutadas posteriormente.

---

### 3.6 Clase Combate

Es la clase central del sistema.

Administra:

- Jugador.
- Enemigos.
- Turnos.
- Acciones.

#### Atributos

```
private JugadorCombate jugador;  
private List<Enemigo> enemigos;  
private Queue<Personaje> turnos;  
private Stack<Accion> acciones;
```

---

## 4. Uso de Estructuras de Datos

### Lista

Implementación:

```
List<Enemigo> enemigos
```

Propósito:

Almacenar todos los enemigos participantes del combate.

Permite:

- Agregar enemigos.

- Recorrer enemigos.
  - Eliminar enemigos derrotados.
- 

## Cola

Implementación:

`Queue<Personaje> turnos`

Propósito:

Administrar el orden de los turnos.

Se utiliza una política FIFO (First In First Out).

Operaciones principales:

`offer()`

`poll()`

Funcionamiento:

El personaje que lleva más tiempo esperando obtiene el siguiente turno.

---

## Pila

Implementación:

`Stack<Accion> acciones`

Propósito:

Almacenar las acciones seleccionadas por el jugador.

Se utiliza una política LIFO (Last In First Out).

Operaciones principales:

`push()`

`pop()`

Funcionamiento:

La última acción agregada es la primera en ejecutarse.

---

## 5. Flujo del Combate

### Inicio

1. Se crea el jugador.
  2. Se agregan enemigos.
  3. Se inicializa la cola de turnos.
- 

### Selección de acciones

El usuario puede agregar acciones mediante los botones de la interfaz.

Cada acción se almacena en:

`acciones.push(...)`

---

### Procesamiento del turno

Cuando se procesa un turno:

1. Se obtiene el personaje actual desde la cola.
  2. Si es el jugador, se ejecutan las acciones almacenadas.
  3. Si es un enemigo, se aplica daño al jugador.
  4. Si continúa con vida, vuelve a ingresar a la cola.
- 

### Finalización

El combate termina cuando:

```
!jugador.estaVivo()
```

o

```
enemigos.isEmpty()
```

---

## 6. Paquete bitmap

### 6.1 Clase BitmapCiudad9

Responsable de representar gráficamente el estado del combate.

#### Funciones principales

```
dibujarEstadoCombate(...)
```

```
dibujarVictoria()
```

```
dibujarDerrota()
```

#### Información mostrada

- Nombre del jugador.
  - Vida del jugador.
  - Lista de enemigos.
  - Vida de cada enemigo.
  - Mensajes de victoria o derrota.
- 

## 7. Paquete vista

### 7.1 Clase PanelCiudad9

Panel encargado de mostrar la imagen generada por BitmapCiudad9.

#### Responsabilidades

- Actualizar el estado visual.

- Mostrar victoria.
  - Mostrar derrota.
- 

## 7.2 Clase VentanaCiudad9

Ventana principal de la ciudad.

### Componentes

- Botón Ataque.
- Botón Defensa.
- Botón Habilidad.
- Botón Procesar Turno.
- Botón Salir.

### Responsabilidades

- Recibir acciones del usuario.
  - Interactuar con la clase Combate.
  - Actualizar la interfaz.
- 

## 8. Posibles Mejoras Futuras

- Incorporar distintos tipos de enemigos.
  - Permitir seleccionar objetivos específicos.
  - Agregar efectos visuales de combate.
  - Incorporar niveles de dificultad.
  - Añadir sistema de experiencia y progresión del jugador.
- 

## 9. Conclusión



La Ciudad 9 implementa un sistema de combate por turnos utilizando las estructuras de datos requeridas por el trabajo práctico. La solución mantiene una separación entre lógica, representación gráfica e interfaz de usuario, facilitando el mantenimiento y la extensión futura del código.

## Ciudad 10 – Teorema Maestro

# 1. Clase Recurrencia

### Descripción

Representa una recurrencia de la forma " $T(n)=aT(n/b)+n^k$ ". Almacena los parámetros necesarios para aplicar el Teorema Maestro.

### Atributos

- `private int a`

Cantidad de subproblemas generados en cada llamada recursiva.

- `private int b`

Factor de reducción del tamaño del problema.

- `private int k`

Exponente de la función de costo adicional.

### Métodos

- `public Recurrencia(int a, int b, int k)`

Constructor que inicializa los tres parámetros.

- `public int getA()`

Devuelve el valor de a.

- `public int getB()`

Devuelve el valor de b.

- `public int getK()`

Devuelve el valor de k.

## 2. Clase `ParserRecurrencia`

### Descripción

Se encarga de interpretar la recurrencia ingresada por el usuario y convertirla en un objeto Recurrencia.

### Métodos

- `public Recurrencia parsear(String texto)`

Recibe el texto ingresado por el usuario.

Si el formato es válido devuelve una Recurrencia.

Si no lo es devuelve null.

## 3. Clase `AnalizadorMaestro`

### Descripción

Implementa el Teorema Maestro. Determina automáticamente cuál de los tres casos corresponde a una recurrencia y calcula la complejidad temporal resultante.

### Métodos

- `public String analizar(Recurrencia r)`

Analiza la recurrencia y devuelve:

- Caso detectado.
- Valor de  $\log_b(a)$ .
- Complejidad asintótica.

- `public int obtenerCaso(Recurrencia r)`

Devuelve según el caso que se haya conseguido por parte del usuario.

## 4. Clase `NodoExpansion`

### Descripción

Representa un nivel dentro del árbol de recurrencia. Cada objeto almacena la información necesaria para dibujar un nivel de la expansión.

### Atributos

- `private int nivel`

Número de nivel dentro del árbol.

- `private int cantidadProblemas`

Cantidad de subproblemas existentes en ese nivel.

- `private String tamaño`

Tamaño de los subproblemas del nivel.

### Métodos

- `public NodoExpansion(int nivel, int cantidadProblemas, String tamaño)`

Constructor.

- `public int getNivel()`

Devuelve el nivel.

- `public int getCantidadProblemas()`

Devuelve la cantidad de problemas.

- `public String getTamaño()`

Devuelve el tamaño asociado.

## 5. Clase `ExpansorRecurrencia`

### Descripción

Genera los niveles de expansión de una recurrencia. Produce la información necesaria para construir el árbol visual.

### Métodos

- `public List<NodoExpansion> expandir(Recurrencia recurrencia,int niveles)`

Genera una lista de niveles. Para cada nivel calcula:

$a^i$  problemas y  $n/b^i$  donde  $i$  representa el nivel actual.

Devuelve una lista de `NodoExpansion`.

## 6. Clase `CondicionVictoria`

### Descripción

Controla el progreso de la Ciudad 10. Registra cuáles de los tres casos del Teorema Maestro ya fueron descubiertos por el usuario.

### Atributos

- `private boolean caso1`

Indica si el Caso 1 fue encontrado.

- `private boolean caso2`

Indica si el Caso 2 fue encontrado.

- `private boolean caso3`

Indica si el Caso 3 fue encontrado.

### Métodos

- `public void completarCaso(int caso)`

Marca un caso como descubierto.

- `public boolean casoCompleto(int caso)`

Indica si un caso ya había sido registrado.

- `public boolean ciudadCompletada()`

Devuelve true cuando los tres casos fueron encontrados.

- `public String estado()`

Devuelve el estado actual del progreso.

## 7. Clase `PanelArbolRecurrencia`

### Descripción

Se encarga de generar la representación gráfica del árbol de recurrencia. Utiliza la clase `Bitmap` para dibujar nodos, conexiones y textos. Cuando la cantidad de nodos supera un límite razonable, reemplaza el dibujo completo por una representación resumida para mantener la legibilidad.

### Atributos

- `private List<NodoExpansion> nodos`

Lista de niveles a representar.

- `private Bitmap bmp`

Bitmap donde se dibuja el árbol.

- `private JLabel lblImagen`

Componente visual que muestra el bitmap.

- `private int a`

Parámetro a de la recurrencia actual.

- `private int b`

Parámetro b de la recurrencia actual.

- `private int k`

Parámetro k de la recurrencia actual.

### Métodos

- `public PanelArbolRecurrencia()`

Inicializa el panel gráfico.

- `public void setRecurrencia(int a,int b,int k)`

Guarda los parámetros de la recurrencia actual.

- `public void setNodos(List<NodoExpansion> nodos)`

Actualiza los niveles del árbol y solicita el redibujado.

- `private void dibujarArbol()`

Genera toda la representación gráfica.

Dibuja:

- Nodos.
- Conexiones.
- Tamaños de subproblemas.
- Mensajes explicativos.

Si el árbol supera el límite establecido muestra:

“Los nodos no continúan para mantener legible el gráfico.

Patrón detectado:

Nivel  $i \rightarrow a^i$  problemas de tamaño  $n/b^i$ ”

## 8. Clase `VentanaCiudad10`

### Descripción

Es la interfaz principal de la Ciudad 10. Coordina la interacción entre el usuario y toda la lógica del Teorema Maestro. Permite:

- Ingresar recurrencias.
- Analizarlas.
- Visualizar resultados.
- Dibujar el árbol.
- Controlar la condición de victoria.

### Atributos

- `private JTextField campoRecurrencia`  
Campo donde se escribe la recurrencia.
- `private JTextArea resultado`  
Área donde se muestran los resultados del análisis.
- `private JButton botonAnalizar`  
Botón que inicia el análisis.
- `private PanelArbolRecurrencia panelArbol`  
Panel gráfico del árbol.
- `private CondicionVictoria progreso`  
Controlador del avance del jugador.

## Métodos

- `public VentanaCiudad10()`  
Construye toda la interfaz gráfica. Inicializa:
  - Instrucciones.
  - Campo de texto.
  - Área de resultados.
  - Árbol de recurrencia.
- `private void analizar()`  
Método principal de ejecución.